CHAPTER NO:	TITLE	PAGE NO:
	ABSTRACT	**5**
	LIST OF TABLES	**6**
	LIST OF FIGURES	**7**
	LIST OF ABBREVIATIONS	**10**
1	**INTRODUCTION**	**12**
	1.1 OVERVIEW OF THE PROJECT	13
	1.2 LITERATURE SURVEY	3
	1.2.1 AI-Based Recruitment and Resume Matching Systems	3
	1.2.2 Intelligent Skill Extraction and Resume Parsing Systems	3
	1.2.3 AI-Powered Candidate Ranking and ATS Systems	3-4
	1.2.4 Virtual Technical Interview and Proctored Screening Systems	4
	1.2.5 AI-Driven Skill Gap Analysis and Career Advisory Systems	4
	1.3 PROPOSED SYSTEM OBJECTIVES AND SCOPES	5
	1.3.1 Objectives of the Proposed System	5
	1.3.2 Scope of the Proposed System	5-6
2	**REQUIREMENTS SPECIFICATIONS**	**7**
	2.1 OVERALL DESCRIPTION	8
	2.1.1 Functional Requirements Overview	8-10
	2.2 PRODUCT PERSPECTIVE	10-11
	2.3 PRODUCT FUNCTIONS	11
	2.3.1 Resume Matcher Configuration	11
	2.3.2 AI-Based Resume Matching & Sentence-BERT Scoring	12

	2.3.3 Multimodal Resume Document Submission (PDF/DOCX)	12
	2.3.4 Real-Time Resume Skill Parsing & Quality Scoring	12
	2.3.5 Intelligent Job & Candidate Recommendations	12-13
	2.3.6 ATS Leaderboard Reporting & Skill Gap Feedback	13
	2.3.7 3-Slot Interview Scheduling & Proctored Screening	13
	2.3.8 User Account and Application History Management	13-14
	2.3.9 Admin Monitoring and System Management	14
	2.4 USER CHARACTERISTICS	14
	2.4.1 Master Admin	14
	2.4.2 Candidate (Job Seeker)	15
	2.4.3 Recruiter (Employer)	15
	2.5 OPERATING ENVIRONMENT CONSTRAINTS	15
	2.5.1 Software Specification	16
	2.5.2 Hardware Specification	16
	2.6 SPECIFIC REQUIREMENTS	17
	2.6.1 Candidate Requirements	17
	2.6.2 Recruiter & Admin Requirements	18
	2.6.3 Functional Requirements	18
	2.6.4 Non-Functional Requirements	18-19
	2.6.5 Security Requirements	19
	2.7 EXTERNAL INTERFACE REQUIREMENTS	19-21
	2.8 SYSTEM FEATURES	21-24
3	**SYSTEM DESIGN AND TEST PLAN**	**25-26**
	3.1 DATA FLOW DIAGRAM (DFD)	26
	3.1.1 Level 0 DFD	27-28
	3.1.2 Level 1 DFD	28-29

	3.2 USE CASE DIAGRAM	30
	3.2.1 Master Admin Use Case Diagram	30-31
	3.2.2 Candidate Use Case Diagram	31-32
	3.2.3 Recruiter Use Case Diagram	32-33
	3.3 TABLE DESIGNS	33
	3.3.1 User Table (auth_user & UserProfile)	33-34
	3.3.2 Candidate Profile Table	34-35
	3.3.3 Recruiter Profile Table	35
	3.3.4 Resume Document Table	35-36
	3.3.5 Job Posting Table	36
	3.3.6 Match Result Table	36-37
	3.3.7 Job Application Table	37
	3.3.8 Direct Messaging Table	37-38
	3.3.9 Proctored Interview Session Table	38-39
	3.4 PERFORMANCE REQUIREMENTS	39-40
	3.5 SOFTWARE QUALITY ATTRIBUTES	40-42
	3.6 DETAILED DESIGN	42
	3.6.1 System Architecture	42-43
	3.6.2 User Registration and Authentication	43
	3.6.3 Candidate Profile and Resume Management	43-44
	3.6.4 AI-Based Resume-Job Matching and ATS Ranking	44
	3.6.5 Resume Skill Extraction and Text Processing	44-45
	3.6.6 ATS Reporting and Skill Gap Analytics	45
	3.6.7 Security Measures	45-46
	3.6.8 Performance Optimization	46
	3.6.9 Testing and Deployment	47
	3.7 TEST CASES	47-49
	3.7.1 Test Cases for Admin	49-50

	3.7.2 Test Cases for Candidate	50-51
	3.7.3 Test Cases for Recruiter	51-52
4	**IMPLEMENTATION AND RESULTS**	**53**
	4.1 IMPLEMENTATION	53
	4.1.1 Backend Development	53-54
	4.1.2 Frontend Development	54
	4.1.3 Integration and Testing	55-56
	4.2 IMPLEMENTATION RESULTS	56-57
5	**CONCLUSION AND FUTURE WORKS**	**58**
	5.1 CONCLUSION	58-59
	5.2 FUTURE WORKS	59
	5.2.1 Future Implementation Plan	59-61
	REFERENCES	**62**
	APPENDIXES	**63**
	Appendix A - Screenshots	63-65
	Appendix B - Sample Code	66-96

ABSTRACT

Connecting job seekers with appropriate job vacancies is a major challenge in modern recruitment, largely due to unstandardized resume formats and the limitations of traditional keyword-based search systems. To address this problem, this project proposes **JobFit AI**: an Intelligent Resume Matching and Skill Recommendation Platform integrated with Proctored AI Screening Interviews. The system leverages Natural Language Processing (NLP) and Deep Learning techniques embedded within a web-based human resource platform to automate candidate evaluation, measure true semantic job suitability, and streamline candidate-recruiter interactions.JobFit AI extracts technical competencies, education, and work history from candidate resume documents (PDF/DOCX) using spaCy NLP PhraseMatchers and document parsers. Resume contextual meaning is encoded into 384-dimensional dense vector embeddings via **Sentence-BERT (SBERT)**, allowing Cosine Similarity computations to determine deep semantic compatibility between candidate resumes and Job Descriptions (JDs). Based on prediction scores, candidates receive a Resume Quality Score (0–100), ranked job recommendations, and targeted skill gap analyses paired with recommended online learning courses.For recruiters, the system features an Applicant Tracking System (ATS) dashboard to track candidate application statuses (Applied, Under Review, Shortlisted, Hired, Rejected) and a candidate leaderboard sorted by match percentage. Recruiters can propose 3 interview time slots, initiating an automated AI Mock Screening Interview. The interview room features automated technical question generation, real-time proctoring with JavaScript DOM tab-switch listeners, and automated evaluation scorecards. The platform is developed using Python and Django MVT web framework, SQLite relational database, and Bootstrap 5 UI. By integrating hybrid AI semantic matching, skill gap recommendation, automated screening, and proctored interview evaluation, JobFit AI reduces hiring delays, eliminates keyword bias, and demonstrates effective AI integration in human resource management.

LIST OF TABLES

TABLE NO:	TITLE	PAGE NUMBER
3.1	auth_user	45
3.2	jobfit_candidateprofile	46
3.3	jobfit_recruiterprofile	47
3.4	jobfit_resumedocument	47
3.5	jobfit_jobposting	48
3.6	jobfit_jobapplication	49
3.7	jobfit_interviewschedule	49
3.8	jobfit_proctoredscorecard	50
3.9	jobfit_directmessage	51

LIST OF FIGURES

FIGURE NO:	TITLE	DESCRIPTION / DIAGRAM TYPE
3.1	DFD Level 0	High-level system context diagram for JobFit AI platform
3.2	DFD Level 1	Detailed data flow process diagram for resume parsing, AI vector matching, and proctored interview
3.3	Admin Use Case Diagram	Use case interactions for Master Administrator (Recruiter Approval, Company Management, Safety Reports)

3.4	Job Seeker Use Case Diagram	Use case interactions for Candidate (Upload Resume, View Match Results, Skill Gap, AI Interview)
3.5	Recruiter Use Case Diagram	Use case interactions for Employer Recruiter (Post Job, ATS Applicant Tracking, Slot Proposal, Direct Chat)
A.1	Home Page	Platform landing page introducing JobFit AI capabilities
A.2	Login & Registration Page	User authentication and role-based signup interface
A.3	Upload Resume Page	Candidate resume dropzone and spaCy NLP parser trigger
A.4	Resume Detail & Quality Score	Parsed resume profile, extracted skill badges, and score out of 100
A.5	Match Results Page	AI-ranked job openings list sorted by Sentence-BERT match percentage
A.6	Skill Recommendations Page	Career growth skill gap analysis and recommended online learning courses
A.7	Browse Jobs Page	Job search catalog with category filters and View Companies option

A.8	Single Job Detail Page	Full job description, required skills, and candidate application button
A.9	Candidate Dashboard	Job seeker activity hub tracking applications and interview invites
A.10	Recruiter Dashboard & ATS	ATS tracking table, Candidate Match Leaderboard, and Posted Jobs Management
A.11	Master Admin Panel	Superuser portal for recruiter verification, company profiles, and safety reports
A.12	Proctored AI Screening Room	Live proctored interview room with countdown timer and tab switch listener
A.13	Interview Transcript Page	Post-interview evaluation scorecard, Q&A transcript, and AI feedback
A.14	Messages Inbox & Direct Chat	Direct messaging thread with unread count badge and soft delete option
A.15	User Profile Page	Account settings, username editing, active resume profile, and delete account modal

LIST OF ABBREVIATIONS

LIST	ABBREVIATIONS	MEANING

1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	ATS	Applicant Tracking System
4	BERT	Bidirectional Encoder Representations from Transformers
5	**CSS**	Cascading Style Sheets
6	DFD	Data Flow Diagram
7	DOM	Document Object Model
8	HTML	Hypertext Markup Language
9	HTTP	Hypertext Transfer Protocol
10	JD	Job Description
11	JS	JavaScript
12	MVT	Model-View-Template
13	NLP	Natural Language Processing
14	ORM	Object-Relational Mapping
15	PDF	Portable Document Format
16	RBAC	Role-Based Access Control
17	SBERT	Sentence Bidirectional Encoder Representations from Transformers
18	UI	User Interface
19	URL	Uniform Resource Locator
20	UX	User Experience

CHAPTER 1: INTRODUCTION

In today's rapidly evolving digital recruitment environment, the demand for intelligent job matching and automated talent acquisition systems has increased significantly due to the growing volume of job seekers, competitive hiring demands, and the necessity for faster, unbiased recruitment decision-making. Traditional talent acquisition and employment management platforms often face severe challenges such as keyword-only search limitations, unstandardized resume document parsing, delayed candidate evaluations, inefficient application handling, and a lack of personalized skill development guidance for candidates. To address these critical challenges, **JobFit AI** has been developed as an intelligent web-based human resource platform that integrates artificial intelligence, natural language processing (NLP), deep learning vector matching, and applicant tracking technologies into a unified ecosystem.

The proposed platform is designed to provide users with a smart recruitment environment where candidates (job seekers), recruiters (employers), and platform administrators can interact efficiently through a centralized digital system. The platform utilizes advanced NLP PhraseMatchers and Deep Learning sentence transformers (Sentence-BERT) to extract technical competencies from candidate resume documents (PDF/DOCX) and analyze contextual semantic compatibility between candidates and Job Descriptions (JDs). By processing unstructured resume text and comparing 384-dimensional vector embeddings, the system generates automated Resume Quality Scores (0–100), ranked job recommendations, and technical skill gap analyses that assist candidates in understanding their qualifications and career trajectory.

In addition to AI-based resume matching and skill recommendation, the platform incorporates comprehensive recruitment management functionalities such as an Applicant Tracking System (ATS) dashboard, recruiter company registration, job posting management, 3-slot interview time scheduling, proctored AI screening interviews with browser tab-switch detection, and direct internal messaging. The integration of AI-driven analytics with recruitment workflow management creates a smart ecosystem that enhances talent acquisition accessibility, improves operational efficiency, and supports better hiring decisions.

The system is developed using Python and the Django MVT framework, incorporating machine learning libraries such as spaCy, Sentence-Transformers, PyTorch, and Scikit-Learn. The platform

aims to deliver a secure, scalable, and user-friendly human resource solution capable of supporting multiple user roles and real-time interactions. By combining intelligent prediction algorithms with modern recruitment management workflows, the project serves as an advanced human resource support platform that bridges the gap between traditional job portals and AI-powered digital recruitment systems.

1.1 OVERVIEW OF THE PROJECT

JobFit AI is a comprehensive human resource and recruitment platform designed to improve candidate evaluation, semantic job matching, interview proctoring, and recruitment workflow management through intelligent technologies. The project integrates machine learning models, applicant tracking modules, and web-based communication systems into a single unified platform supporting candidates, recruiters, and administrators.

The system enables job seekers to register securely, upload unstructured PDF or Word resumes, and receive AI-generated technical skill extractions, resume quality grades, and semantic match rankings against active job postings. Deep learning models analyze candidate data to compute contextual similarity scores and identify missing technical skills, generating personalized career development insights and course recommendations.

The recruitment management component allows recruiters to post job descriptions, manage company profiles, filter candidates by match percentage, and update application statuses across five structured ATS stages: Applied, Under Review, Shortlisted, Hired, and Rejected. Recruiters can propose three candidate interview time slots and initiate proctored AI screening interviews. During interviews, live JavaScript DOM event listeners monitor candidate browser tab switches to prevent cheating, automatically terminating sessions after three violations.

Furthermore, the platform focuses on privacy and safety by implementing role-based access control (RBAC), direct message soft deletion ("Delete for Me"), unread notification navbar badges, and Admin Audit Privacy Mode to protect message text during administrative reviews. Overall, JobFit AI serves as an innovative talent acquisition platform that enhances hiring efficiency, eliminates keyword bias, and supports timelier recruitment decisions.

1.2 LITERATURE SURVEY

1.2.1 AI-Based Resume Parsing and Automated Candidate Screening System

The study “AI-Based Resume Parsing and Automated Candidate Screening System" (2025) explores the development of an intelligent talent acquisition platform designed to assist job seekers and recruitment professionals through artificial intelligence and natural language processing. The system enables users to upload unstructured resume documents to extract technical skills, contact details, and qualifications automatically. By integrating spaCy NLP PhraseMatchers, applicant tracking modules, and candidate filtering algorithms, the platform provides early skill assessment and automated candidate ranking. Developed using Python and Django framework, the study highlights the effectiveness of AI-driven recruitment systems in improving hiring accessibility, reducing evaluation delays, and supporting efficient talent management.

1.2.2 Intelligent Semantic Job Matching using Sentence-BERT Embeddings

The study "Intelligent Semantic Job Matching using Sentence-BERT Embeddings" (2025) investigates the development of an AI-powered job recommendation engine that combines deep learning semantic vector embeddings with technical skill overlap mechanisms. The system utilizes Sentence-BERT (SBERT) and PyTorch vector models to analyze resume text and job descriptions, generating contextual similarity scores with Cosine Similarity algorithms. Additionally, the platform recommends specialized job openings and identifies technical skill gaps. Developed using Python and machine learning libraries, the study demonstrates how semantic job matching systems improve candidate guidance, eliminate rigid keyword search constraints, and support personalized hiring.

1.2.3 AI-Powered Virtual Interview Screening and Proctoring Systems

The study "AI-Powered Virtual Interview Screening and Proctoring Systems" (2026) analyzes the growing role of artificial intelligence in assisting candidate screening and remote evaluation processes. The research highlights the limitations of traditional interview scheduling, including delayed candidate feedback, unproctored assessment risks, and scheduling conflicts. By examining modern AI evaluation tools, the study emphasizes the importance of automated question

generation, real-time JavaScript DOM tab-switch listeners, and automated evaluation scoring. The paper discusses how AI-powered screening systems enhance recruitment integrity, reduce screening overhead, and support early technical evaluation.

1.2.4 Applicant Tracking Systems (ATS) and Recruitment Workflow Platforms

The study "Applicant Tracking Systems (ATS) and Recruitment Workflow Platforms" (2024) investigates the effectiveness of digital recruitment platforms in improving recruiter-candidate communication, pipeline management, and applicant status tracking. The research analyzes how ATS platforms support multi-stage application tracking (Applied, Under Review, Shortlisted, Hired, Rejected) and direct messaging through web-based technologies. Using real-time workflow analytics, the study found that integrated ATS platforms significantly improve recruitment turnaround times, reduce communication friction, and enhance overall candidate experience.

1.2.5 AI-Driven Skill Gap Analysis and Career Recommendation Engine

The study "AI-Driven Skill Gap Analysis and Career Recommendation Engine" (2025) explores the use of machine learning techniques in identifying professional skill deficiencies and providing targeted career development guidance. The research highlights that traditional job portals focus solely on present qualifications rather than continuous career growth. The proposed system utilizes NLP skill extraction algorithms to compare candidate profiles against market requirements, detect technical skill gaps, and generate AI-based course recommendations. Through predictive evaluation, the study demonstrates the effectiveness of AI-driven recommendation platforms in boosting candidate employability.

1.3 PROPOSED SYSTEM OBJECTIVES AND SCOPE

1.3.1 Objectives of the Proposed System

The primary objective of the **JobFit AI** project is to develop an intelligent, secure, and interactive talent acquisition platform that assists job seekers, recruiters, and administrators through advanced artificial intelligence, natural language processing, and recruitment management technologies. The system aims to provide a user-friendly environment where candidates can upload resumes to receive AI-based quality scores, semantic job match rankings, and skill gap analyses. By

integrating deep learning vector transformers (Sentence-BERT), the platform seeks to eliminate keyword bias and improve hiring decision accuracy.

Furthermore, the system is designed to enhance recruitment operational efficiency by incorporating functionalities such as 3-slot interview scheduling proposals, proctored AI mock screening interviews with browser tab-switch violation tracking, ATS applicant status updates, direct internal messaging, and role-based access control. The platform evaluates candidate suitability using hybrid prediction models capable of analyzing resume text and generating match percentage badges (0–100%). Overall, the proposed system strives to bridge the gap between traditional job portals and modern AI-powered digital recruitment platforms.

1.3.2 Scope of the Proposed System

The scope of the **JobFit AI** project includes the design and development of an intelligent web-based recruitment platform that integrates AI resume parsing, semantic vector matching, proctored screening interviews, and comprehensive ATS management workflows. The system supports candidates, recruiters, and administrators by providing centralized recruitment services and AI-driven candidate evaluation.

The platform enables candidates to access resume analysis modules across IT & Software, Data Science & AI, DevOps & Cloud, Product Management, Design & UI/UX, and Marketing categories. Users can view parsed skills, receive resume quality scores, track submitted job applications, and attend proctored AI screening interviews. Recruiters can post job listings, rank candidate resumes on a leaderboard, propose 3 interview time slots, update candidate ATS statuses, and send direct messages.

Additionally, the platform provides administrative controls for recruiter verification (is_approved), company profile management, safety violation report resolution, and message privacy audit logging. Future extensions of the project scope include video-based AI emotion analysis, automated resume builder integration, multilingual support, cloud deployment, and real-time candidate assessment analytics. Overall, the system serves as a scalable, intelligent, and extensible talent acquisition solution.

CHAPTER 2 : REQUIREMENTS SPECIFICATIONS

The requirement specification for the "JobFit AI" project outlines the development of an intelligent and interactive web-based recruitment and resume-job matching platform using modern

technologies such as Python, Django framework, deep learning models, and integrated natural language processing (NLP) libraries. The system is designed to assist job seekers, recruiters/employers, and administrators by providing AI-based resume parsing, semantic job-fit scoring, skill gap recommendations, applicant tracking (ATS), and proctored AI screening interviews within a centralized digital environment. It incorporates advanced technologies including Sentence-BERT (SBERT) vector embeddings for contextual semantic matching, spaCy NLP for technical skill extraction, and real-time JavaScript DOM listeners for interview proctoring.Key functionalities of the system include secure user authentication and profile management, automated resume document parsing, AI-driven job match score generation, skill gap identification, 3-slot interview scheduling, live proctored screening interviews, transcript evaluation scorecards, internal direct messaging, and centralized administrative oversight dashboards. The platform enables job seekers to upload resumes in PDF or DOCX formats and receive an automated Resume Quality Score (0–100) along with extracted skill badges and matched job recommendations ranked by semantic similarity. Deep learning models analyze resume text and job descriptions to generate match confidence percentages along with actionable career advice.The platform also supports efficient interaction between job seekers, recruiters, and administrators through dedicated management modules. Recruiters can post job openings using single-viewport compact forms, review top-matched candidates via a ranked leaderboard, manage applicant statuses (Applied -> Under Review -> Shortlisted -> Hired / Rejected), propose 3 interview time slots, and evaluate proctored interview transcripts. Job seekers can track application statuses, select preferred interview time slots, participate in proctored technical screening interviews, and receive direct recruiter messages.Additionally, the system includes comprehensive reporting and evaluation functionalities such as interview transcript scorecards, question-by-question scoring, tab-switch proctoring logs, application history tracking, and internal messaging audit oversight. Real-time notifications and message indicators improve communication across the platform. The admin module is responsible for monitoring platform activities, verifying recruiter registrations, managing employer company profiles, analyzing system-wide evaluation trends, and ensuring platform security and reliability.Overall, JobFit AI aims to deliver a scalable, secure, and user-friendly recruitment solution that bridges the gap between job seekers and modern digital

recruitment technologies, thereby improving hiring accessibility, supporting objective candidate screening, enhancing interview integrity, and promoting intelligent recruitment decision-making.

2.1 OVERALL DESCRIPTION

The JobFit AI project is a comprehensive web-based recruitment application developed using modern technologies such as Python and the Django framework, integrated with advanced artificial intelligence and natural language processing models. The platform is designed to provide an intelligent recruitment environment that assists candidates, recruiters, and administrators in managing hiring services efficiently. It offers a robust backend system that manages resume parsing, semantic matching, skill gap analysis, interview scheduling, proctored AI evaluations, and internal messaging while ensuring secure and reliable data processing. In the rapidly evolving digital hiring sector, the system addresses the growing need for an efficient and intelligent recruitment platform where job seekers can access AI-ranked job recommendations and recruiters can identify top talent conveniently. The platform enables candidates to upload resumes, receive AI-generated fit scores, and access skill gap recommendations in real time. By incorporating features such as vector-based similarity matching, proctored screening interviews, candidate leaderboards, and resume history management, the platform ensures a comprehensive and user-friendly hiring experience while supporting fair evaluation and informed recruitment decision-making.

2.1.1 Functional Requirements:

The functional requirements for the JobFit AI project are designed to provide a comprehensive and intelligent recruitment management solution for users. The platform enables job seekers, recruiters, and administrators to register securely, log in, and manage profiles, ensuring personalized access to hiring workflows and administrative functionalities. Candidates can access resume upload modules, after which the system analyzes document text and generates AI-based skill extraction and match scores.

To ensure accurate candidate assessment, the system integrates advanced machine learning algorithms for analyzing resume content and job description requirements. Prediction models process resume inputs and generate job fit scores, match confidence levels, and detailed skill gap insights. The system also supports intelligent 3-slot interview scheduling mechanisms that suggest suitable time options for proctored AI screening interviews.The platform provides applicant tracking (ATS) functionalities that allow recruiters to manage candidate pipelines, review prediction reports, monitor evaluation history, update job postings, and track applicant statuses efficiently. Recruiters can manage job openings, review candidate resumes, monitor proctored screening transcripts, and manage candidate communications through dedicated dashboards.The system also includes comprehensive reporting and analytics features such as candidate evaluation history, ATS performance statistics, visual skill gap analytics, and personalized learning course recommendations. Notifications and unread message indicators improve communication between candidates and recruiters while enhancing overall platform accessibility and operational efficiency.To maintain system security and data privacy, the platform incorporates robust authentication and authorization mechanisms, ensuring controlled access for candidates, recruiters, and administrators. The admin module provides a dedicated dashboard for monitoring recruitment activities, verifying recruiter accounts, managing company profiles, and analyzing overall system performance. The backend system efficiently handles communication between frontend interfaces, deep learning models, and relational databases to ensure smooth recruitment operations and secure data management. Furthermore, the system is designed with scalability, reliability, and performance optimization in mind, allowing it to support multiple concurrent users and large volumes of recruitment data efficiently. The platform is responsive across desktops, tablets, and smartphones, ensuring accessibility for users on multiple devices. Overall, the functional requirements aim to deliver a secure, intelligent, scalable, and user-friendly recruitment platform that enhances hiring accessibility, supports fair candidate evaluation, and improves recruitment management through AI-driven automation and vector analytics.

2.2 PRODUCT PERSPECTIVE

JobFit AI is positioned as an advanced and intelligent web-based recruitment platform designed to improve candidate matching accessibility, resume evaluation, applicant tracking, and screening

coordination through the integration of artificial intelligence, natural language processing, and modern web technologies. The system is developed using Python and the Django framework, providing a scalable, secure, and efficient architecture capable of supporting recruitment operations, predictive analytics, and real-time proctored evaluation within a centralized digital platform environment.At the core of the system, the relational database is utilized to efficiently manage candidate profiles, recruiter accounts, company information, job listings, application pipelines, interview sessions, QA scorecards, and internal message threads. The structured database design ensures secure data storage, fast retrieval, reliable record management, and efficient tracking of candidate evaluation history over time. The Django framework provides a robust backend infrastructure that manages authentication, routing, session handling, recruitment workflows, and communication between AI models and frontend interfaces, ensuring smooth and reliable system operation.

The platform integrates advanced artificial intelligence and machine learning technologies to deliver intelligent recruitment functionalities. Sentence-BERT deep learning models are utilized for semantic vector similarity matching by analyzing resume text and job descriptions. Predictive vector algorithms generate match confidence scores and skill gap recommendations. Proctored AI interview modules are also integrated to screen candidates dynamically while JavaScript DOM event listeners monitor tab switching to ensure test integrity. From an interaction perspective, the system provides an intuitive and user-friendly interface where candidates can upload resumes, access match results, view skill gap recommendations, and participate in screening interviews efficiently. Recruiters are provided with dedicated dashboards for managing ATS pipelines, reviewing candidate transcripts, updating job postings, and managing interview invitations. Functionally, the system supports secure user authentication, AI-driven resume parsing, match scoring, applicant tracking, candidate history management, and screening reporting functionalities. It also enables continuous performance tracking, intelligent prediction analysis, personalized learning guidance, and real-time messaging to support improved hiring decisions and recruitment efficiency. In terms of security and reliability, the platform incorporates strong authentication mechanisms, CSRF protection, 100% ORM parameterized database queries, and secure data management strategies to protect sensitive candidate information and ensure system integrity. The

system is designed to be scalable and adaptable, enabling efficient performance even with increasing recruitment data and concurrent users. Overall, JobFit AI serves as a comprehensive and intelligent hiring solution that combines predictive analytics, applicant tracking, and AI-powered screening to provide users with a modern, efficient, and accessible digital recruitment experience.

2.3 PRODUCT FUNCTIONS

2.3.1 Resume Upload & Parsing Configuration

The Resume Upload & Parsing Configuration functionality allows candidates to initiate personalized career assessments by uploading PDF or DOCX resume documents up to 5MB. The system executes pdfplumber or python-docx text extraction and passes raw unstructured text to spaCy NLP PhraseMatcher for technical skill extraction and contact detail parsing. This ensures that each candidate assessment is relevant, intelligent, and aligned with current market skills.

2.3.2 AI-Based Resume Quality Scoring

The AI-Based Resume Quality Scoring feature enables the system to evaluate candidate resume completeness and formatting quality across multiple parameters. The system processes resume inputs and generates a 0-100 Quality Score with letter grade ratings (A+, A, B, C) and actionable improvement feedback. This functionality supports early resume optimization and helps job seekers enhance profile presentation before applying.

2.3.3 Multimodal Resume & Job Input Submission

The Multimodal Resume & Job Input Submission functionality allows users to provide recruitment information through multiple input methods such as document file upload, text-based job description entry, and structured form field selection. The system supports single-viewport compact forms for entering job details, company profiles, and skill requirements efficiently. This flexibility enables users to interact naturally with the platform while ensuring accurate data collection for AI matching and analysis.

2.3.4 Real-Time Sentence-BERT Vector Matching Engine

The Real-Time Sentence-BERT Vector Matching Engine analyzes candidate resume text and job description requirements using trained Sentence-BERT (all-MiniLM-L6-v2) deep learning models. The system evaluates semantic context to compute dense 384-dimensional vector embeddings and calculate Cosine Similarity scores under a hybrid formula (70% vector similarity + 30% technical skill overlap). This ensures precise, objective, and non-keyword-biased job fit ranking for candidates and recruiters.

2.3.5 Skill Gap Analysis & Learning Path Recommendation

The Skill Gap Analysis & Learning Path Recommendation functionality compares candidate technical competencies against industry job requirements. The system identifies missing skills (such as Docker, Kubernetes, or GraphQL) and presents direct links to curated online learning courses. This feature improves candidate career growth and helps job seekers upskill for target positions.

2.3.6 Automated ATS Reporting and Candidate Scorecard Feedback

The Automated ATS Reporting and Candidate Scorecard Feedback feature provides recruiters and candidates with detailed evaluation reports generated from proctored AI screening interviews. It includes overall evaluation scores %, question-by-question ratings, candidate response logs, AI summary feedback, and recruiter re-attempt controls, helping HR managers evaluate technical competency efficiently.

2.3.7 3-Slot Proctored AI Interview Scheduling & Execution

The 3-Slot Proctored AI Interview Scheduling & Execution functionality allows recruiters to propose 3 alternative interview time slots for candidate screening. Candidates select 1 preferred slot with a single click, unlocking a 3-hour proctored interview window. In the live interview room, JavaScript DOM listeners monitor window blur and tab switching, triggering warnings and auto-locking the session after 3 violations to ensure interview fairness.

2.3.8 User Account and Resume History Management

The User Account and Resume History Management functionality enables candidates, recruiters, and administrators to securely register, log in, and manage profiles. Candidates can access previous resume parsing results, application records, and past interview transcripts while tracking career progress over time. Recruiters can manage company profiles, job listings, and ATS pipelines through centralized dashboards.

2.3.9 Admin Monitoring and Platform Oversight

The Admin Monitoring and Platform Oversight feature provides administrators with tools to oversee recruitment platform operations and monitor overall system performance. Admins can verify recruiter registrations, manage employer company profiles, monitor platform metric counts (Jobs, Resumes, Evaluations), inspect message privacy audit logs, and ensure platform security and reliability.

2.4 USER CHARACTERISTICS

2.4.1 Admin (Master Administrator)

Admins in JobFit AI are responsible for managing and monitoring the overall functioning of the recruitment platform. They have superuser access to the system, allowing them to oversee candidate and recruiter activities, verify employer registrations, manage company profiles, and inspect message audit privacy logs. Admins can analyze platform performance through metric counters and ensure the accuracy and reliability of AI matching models. They are also responsible for platform maintenance, user management, data security, and system performance monitoring.

2.4.2 Job Seeker (Candidate)

Job Seekers are primary users who utilize the system to access job opportunities, resume quality scoring, and AI screening interviews. Candidates can securely register, log in, and manage personal profiles. They can upload resumes in PDF or DOCX format to receive AI-generated Quality Scores, skill badges, and matched job recommendations. Candidates can also search for jobs, apply with 1 click, confirm interview time slots, participate in proctored AI screening sessions, and receive direct recruiter messages.

2.4.3 Recruiter (Employer / HR Manager)

Recruiters in the platform are responsible for creating job postings, reviewing candidate applications, and conducting AI screening interviews. They can post jobs using single-viewport compact forms, review top candidate matches via ranked leaderboards, manage applicant statuses in the 3-tab ATS pipeline, propose 3-slot interview invitations, and evaluate proctored interview scorecards. Recruiters can also grant interview re-attempt permissions and communicate directly with candidates via internal messaging.

2.5 OPERATING ENVIRONMENT CONSTRAINTS

Developing JobFit AI involves the integration of modern web technologies, deep learning models, and recruitment management tools to ensure secure, scalable, and efficient hiring operations. The development environment is designed to support intelligent resume matching, skill gap analytics, ATS management, and real-time proctored interviews through a reliable web platform.

2.5.1 Software Specifications

Operating System: Windows 10/11, Ubuntu 20.04+, or macOS

Web Browser: Any Standard Web Browser (Google Chrome, Firefox, Edge)

Frontend: HTML5, CSS3, JavaScript (ES6+), Bootstrap 5.3

Backend: Python 3.8+ & Django Framework (4.x / 5.x)

Database: SQLite3 (Relational Database)

Programming Language: Python 3.8+

AI & NLP Libraries: Sentence-BERT (sentence-transformers), PyTorch, spaCy, pdfplumber, python-docx

AI Model Integration: PyTorch & Joblib

Development Environment: VS Code / Antigravity IDE

Version Control: Git & GitHub

2.5.2 Hardware Specifications

Processor: Intel i5 / AMD Ryzen 5 or higher

RAM: 8GB Minimum

Storage: 1GB+ Free Disk Space

Display: Standard Monitor with 1366x768 Resolution or Higher

Peripherals: Keyboard and Mouse

2.6 SPECIFIC REQUIREMENTS

2.6.1 User Requirements:

The user requirements of the system focus on providing secure and personalized recruitment services for candidates and recruiters. The system must allow users to register and authenticate securely using credentials such as email and password while ensuring password encryption and secure session handling. A password recovery mechanism must be available for users who forget login details. Candidates should be able to manage profiles, upload PDF/DOCX resumes, access AI-generated fit scores, search jobs, apply with 1 click, confirm interview slots, take proctored screening tests, and send internal messages. Recruiters must be able to post jobs, review candidate match leaderboards, update ATS applicant statuses, propose 3-slot interview invitations, evaluate proctored scorecards, and manage company profiles.

2.6.2 Admin Requirements:

The admin requirements focus on providing complete control and monitoring capabilities for recruitment platform management. Administrators must be able to monitor platform activities, verify recruiter signups, manage user accounts, and oversee system performance. The admin module should provide analytical dashboards that display platform metrics (Active Jobs, Registered Companies, Uploaded Resumes, AI Evaluations) and user activity logs. Admins must

also be capable of inspecting message privacy audit logs while protecting confidential conversation details and maintaining system security and reliability.

2.6.3 Functional Requirements:

The functional requirements define the major hiring operations and intelligent functionalities supported by the platform. The system must provide secure authentication and role-based access control. It should support resume text parsing (pdfplumber), spaCy NLP skill extraction, and Sentence-BERT Cosine Similarity vector matching. The platform must generate 0-100 Resume Quality Scores, skill gap learning recommendations, ATS applicant status tracking, 3-slot interview scheduling, live tab-switch proctoring, and automated scorecard generation. Efficient communication between frontend interfaces, deep learning models, and SQLite3 database models must be ensured for smooth operations.

2.6.4 Non-Functional Requirements:

The non-functional requirements focus on maintaining high system performance, reliability, scalability, and usability. The platform must support multiple concurrent users without degrading response times (< 1 second for match ranking). It should provide a responsive and user-friendly interface compatible across desktops, tablets, and smartphones. All 10 user input form templates must be optimized into single-viewport compact layouts (100% visible on 1 screen without vertical scrolling). High availability and data integrity must be maintained through parameterized ORM database queries and clean error handling.

2.6.5 Security Requirements:

The security requirements are essential for protecting sensitive candidate data and ensuring platform integrity. The system must ensure that data is transmitted securely using HTTPS protocols. Passwords must be hashed and stored securely. Role-based access control must be enforced to restrict functionalities according to candidate, recruiter, and admin roles. Views must strictly verify recruiter company ownership before allowing job edits or deletions. All database queries must use 100% parameterized Django ORM queries to prevent SQL Injection attacks. CSRF token validation must be mandated on all POST form submissions.

2.7 EXTERNAL INTERFACE REQUIREMENTS

The external interface requirements define how JobFit AI interacts with users, hardware components, databases, and software libraries to ensure smooth operations.

User Interface Requirements: Focuses on providing a clean, intuitive Bootstrap 5.3 Dark Navy (#0f172a) and Emerald Green (#10b981) web interface. Forms use single-viewport compact cards (row g-2) fitting 100% on 1 screen without scrolling. Dashboards present applicant tracking tables, match percentage badges, countdown timers, and evaluation scorecards clearly across desktop and mobile screens.

Hardware Interface Requirements: Standard hardware devices such as keyboard, mouse, and monitor with 1366x768 resolution or higher. The system handles vector computations cleanly on standard computing processors without requiring dedicated external hardware.

Software Interface Requirements: Backend Django ORM connects to SQLite3 database. Deep learning PyTorch models execute Sentence-BERT embeddings. spaCy NLP PhraseMatcher handles skill parsing, while pdfplumber and python-docx extract raw document text.

Communication Interface Requirements: Standard HTTP/HTTPS protocols for encrypted data transfer. Real-time unread message counter badges in the navbar notify users of direct messages and interview invitations.

2.8 SYSTEM FEATURES

AI-Based Semantic Matching: Sentence-BERT vector Cosine Similarity combined with spaCy technical skill overlap ratios to evaluate true contextual job fit beyond keyword searching.

Interactive Recruitment Portal: Responsive dashboards for Job Seekers (resume score, matches, interview invites), Recruiters (ATS status pipeline, candidate leaderboard, job management), and Master Admins.

Skill Gap Analytics: Automated identification of missing technical competencies and recommendation of targeted online learning courses for career growth.

Real-Time Evaluation Scorecard Generation: Post-interview transcript logging, question-level scores, AI evaluation % summaries, and recruiter re-attempt controls.

3-Slot Interview Scheduling: Recruiters offer 3 alternative interview time options; candidates confirm 1 slot with a single click.

Proctored AI Screening Room: Dynamic role-tailored technical questions, live countdown timer, and JavaScript DOM (blur/visibilitychange) tab-switch anti-cheat monitoring with auto-lock after 3 violations.

User Profile & Resume History Management: Allows users to manage profile photos, phone numbers, passwords, active resume links, or request account deletion via confirmation modals.

Admin Dashboard & Oversight: Superuser portal to verify recruiter signups, manage company listings, monitor platform metrics, and inspect message privacy audit logs.

Security Features: 100% Django ORM parameterized queries for SQL injection protection, CSRF token validation, password hashing, and view-level recruiter job ownership verification.

Notification & Direct Messaging System: Direct messaging threads between candidates and recruiters with navbar unread counters and soft deletion ("Delete for Me").

2.5 OPERATING ENVIRONMENT CONSTRAINTS

Developing JobFit AI involves the integration of modern web technologies, deep learning models, and resume/recruitment analytics tools to ensure secure, scalable, and efficient hiring operations. The development and deployment environment is designed to support automated resume text extraction, semantic match score calculation, skill gap analysis, proctored online screening, and real-time candidate-recruiter messaging. The system architecture combines frontend interface

libraries, backend frameworks, natural language processing libraries, and relational database systems to provide smooth recruitment functionality and accurate candidate evaluation.

2.5.1 Software Specifications

Software requirements are technical specifications necessary for the successful development, deployment, and execution of the JobFit AI recruitment management system. The objective of defining software requirements is to ensure consistent system performance, secure handling of candidate resume data, and full compatibility across user platforms.

- Operating System: Windows 10/11, Ubuntu 20.04+, or macOS
- Web Browser: Google Chrome, Mozilla Firefox, Microsoft Edge, Safari
- Frontend Frameworks & Languages: HTML5, CSS3, JavaScript (ES6+), Bootstrap 5.3
- Backend Framework: Python 3.8+ and Django Web Framework (4.x / 5.x)
- Database System: SQLite3 (Relational Database Management System)
- Programming Language: Python 3.8+
- AI & NLP Libraries: Sentence-BERT (sentence-transformers), PyTorch, spaCy (en_core_web_sm), pdfplumber, python-docx
- Development Environment (IDE): Visual Studio Code / Antigravity IDE
- Version Control System: Git and GitHub Repository Management

2.5.2 Hardware Specifications

Hardware requirements define the physical computing assets needed to run the JobFit AI web application, execute Deep Learning model inferences (Sentence-BERT vector embeddings), and parse uploaded candidate resume documents seamlessly.

- Processor (CPU): Intel Core i5 / AMD Ryzen 5 or higher
- System Memory (RAM): 8 GB Minimum (16 GB Recommended for fast PyTorch tensor computations)
- Disk Storage: 1 GB+ Free Disk Space (for Python environment, AI models, and database storage)

- Display Monitor: 1366 × 768 Resolution or higher (Optimized for responsive Bootstrap layout)
- Input Peripherals: Standard Keyboard, Mouse / Touchpad, and Web Camera / Microphone (for proctored interview room)

2.6 SPECIFIC REQUIREMENTS

Specific requirements encompass the detailed user capabilities, functional specifications, non-functional performance constraints, and platform security standards required for JobFit AI.

2.6.1 User Requirements

1. Candidates (Job Seekers) must be able to register, log in, upload resumes in PDF or DOCX format, view their automated Resume Quality Score (0–100), receive matched job recommendations based on semantic similarity, apply for jobs with 1 click, confirm interview time slots, participate in proctored AI screening interviews, and view recruiter messages.
2. Recruiters (Employers) must be able to submit company registration details, await admin verification, post job vacancies, filter applicants using a ranked match score leaderboard, transition applicant statuses (Applied -> Under Review -> Shortlisted -> Hired/Rejected), offer 3-slot interview options, inspect proctored interview scorecards, and send direct messages to candidates.

2.6.2 Admin Requirements

1. Master Administrators must have superuser access to review and approve pending recruiter account registrations.
2. Admins must be able to verify company profiles, monitor platform-wide metric counters (Total Jobs, Companies, Resumes, and Evaluations), manage platform users, and perform direct message privacy audit checks.

2.6.3 Functional Requirements

1. Resume Text Parsing: Automated text extraction from PDF and DOCX files using pdfplumber and python-docx.

2. NLP Skill Extraction: Extraction of candidate technical skills using spaCy PhraseMatcher against an automated technical skill dictionary.
3. AI Semantic Match Calculation: 384-dimensional vector embedding generation via Sentence-BERT (all-MiniLM-L6-v2) and Cosine Similarity calculation combined with technical skill overlap ratio.
4. ATS Applicant Tracking Pipeline: Real-time applicant status transitions with recruiter oversight.
5. 3-Slot Interview Scheduling: Recruiter selection of 3 alternative datetime slots, with 1-click candidate confirmation unlocking a 3-hour access window.
6. Proctored Screening Room: Dynamic technical interview questions with countdown timer and JavaScript DOM tab-switch detection that auto-locks after 3 warnings.
7. Scorecard & Transcript Generation: Instant logging of candidate Q&A responses, question-level scores, and overall evaluation summaries.

2.6.4 Non-Functional Requirements

1. Performance & Speed: Semantic match calculations and page loads must execute within 1 second.
2. Compact Viewport Layout: All 10 user input forms must fit within a single viewport (100% visible on standard screens without vertical scrolling).
3. Responsiveness: The interface must adapt seamlessly across desktop monitors, laptops, and mobile viewports using Bootstrap 5.3.
4. Usability: Intuitive dark navy (#0f172a) and emerald green (#10b981) aesthetic with real-time navbar unread badges and progress counters.

2.6.5 Security Requirements

1. Authentication & Authorization: Secure Django session management and password hashing (PBKDF2/Argon2).
2. Protection Against Web Vulnerabilities: 100% parameterized Django ORM queries to prevent SQL Injection, and CSRF token validation on all POST forms.

3. Access & Ownership Controls: View-level ownership checks preventing recruiters from editing or deleting job postings owned by other employers.

2.7 EXTERNAL INTERFACE REQUIREMENTS

1. User Interface (UI): Designed using Bootstrap 5.3 with a modern dark theme (#0f172a Navy background, #10b981 Emerald Green primary accents). Features include compact form fields (row g-2), tabbed recruiter management portals, real-time unread badges, and modal popups for account operations.
2. Hardware Interface: Standard computer monitor display, keyboard, mouse/touchpad, and microphone/camera setup for interactive proctored online screening.
3. Software Interface: Python-Django framework interfacing with an SQLite3 relational database via Django ORM, Sentence-BERT transformer models executing via PyTorch framework, spaCy NLP pipeline, and pdfplumber/python-docx file readers.
4. Communication Interface: Secure HTTP/HTTPS network protocol for client-server communication, JSON data transfer for asynchronous dynamic updates, and an integrated direct messaging channel with unread notification badges.

2.8 SYSTEM FEATURES

1. AI-Based Semantic Resume & Job Matching: Generates dense vector embeddings for resumes and job posts using Sentence-BERT, calculating precise Cosine Similarity scores and skill match percentages to rank candidates objectively.
2. Interactive Multi-Role Recruitment Portal: Provides distinct customized dashboards for Job Seekers (resumes, score, matched jobs, application history), Recruiters (ATS pipeline, job posting, 3-slot scheduling, scorecard inspection), and Admins (recruiter verification, company management, message privacy logs).
3. Automated Skill Gap Analysis & Career Recommendations: Identifies missing technical skills by comparing candidate profiles with target job requirements, providing direct online course links for skill enhancement.

4. Real-Time Proctored AI Screening Room: Delivers role-tailored technical questions, live timers, and JavaScript DOM tab-switch anti-cheat monitoring (auto-locking the session after 3 tab switches).
5. 3-Slot Interview Scheduling System: Simplifies scheduling by allowing recruiters to offer 3 date/time choices, allowing candidates to confirm 1 choice with a single click and opening a 3-hour access window.
6. Evaluation Scorecard & Transcript Reporting: Automatically compiles candidate interview Q&A logs, calculates AI evaluation score percentages, and displays summary feedback for recruiter review.
7. User Profile & History Management: Enables users to upload profile pictures, update contact details, change passwords, view application history, or invoke account deletion modal dialogs.
8. Platform Oversight & Admin Governance: Provides administrators with single-click recruiter approval queues, company verification controls, platform analytics, and message audit tracking.
9. Enterprise Data Security & Integrity: Enforces Django ORM SQL parameterization, CSRF token validation, role-based view restrictions, and single-viewport form design standards.
10. Integrated Messaging & Notification System: Facilitates direct candidate-recruiter chat threads with real-time navbar unread counters, notification badges, and soft deletion ("Delete for Me") capabilities.

CHAPTER 3

SYSTEM DESIGN AND TEST PLAN

System design refers to the process of defining and developing the architecture, components, modules, interfaces, workflows, and data management structure of a system based on the requirements identified during the analysis phase. It serves as the foundation for transforming conceptual AI-driven recruitment ideas into a fully functional intelligent software solution. In the JobFit AI project, system design focuses on creating a secure, scalable, intelligent, and efficient recruitment platform capable of supporting resume parsing, AI semantic matching, skill gap analysis, ATS applicant tracking, 3-slot interview scheduling, and proctored screening interviews between job seekers, recruiters, companies, and administrators through the integration of artificial intelligence and natural language processing technologies.

The system design process is developed based on the functional and non-functional requirements identified during recruitment system analysis. It involves planning the overall architecture of the hiring platform, defining the interaction between modules, and establishing the flow of candidate and job data throughout the system. The design ensures that the proposed system fulfills its primary objectives of providing AI-based semantic matching (Sentence-BERT), intelligent skill gap recommendations (spaCy NLP), 3-slot interview scheduling, candidate ranking analytics, and secure recruitment management services. System analysts and developers must clearly understand these objectives to create an efficient and reliable architecture that supports seamless communication between users, deep learning models, relational databases, and administrative modules.

JobFit AI is designed as a web-based recruitment application using Python and the Django framework integrated with deep learning models and document parsing libraries. The system architecture consists of multiple interconnected modules including user authentication, candidate resume management, recruiter ATS management, company profile management, AI semantic match scoring, skill gap analytics, 3-slot interview scheduling, proctored screening room, scorecard generation, notification handling, and admin oversight management. Each module is

designed to perform specific recruitment-related operations while maintaining smooth interaction and efficient data exchange with other system components.

System design also includes the development of a structured database architecture for storing user accounts, candidate profiles, uploaded resume text, company details, job vacancy listings, application statuses, interview schedules, proctored Q&A transcripts, direct messages, and administrative records securely. Proper relational database design (SQLite3) ensures efficient data retrieval, secure storage, data consistency, and scalability for handling increasing candidate registrations and job applications. The backend logic (Django ORM) manages communication between the frontend interface, Sentence-BERT embedding models, spaCy NLP parsers, and SQLite3 database tables, while the frontend provides an intuitive, responsive, and user-friendly single-viewport experience across standard web browsers.

Additionally, the system design incorporates advanced security mechanisms such as encrypted session protocols, role-based access control (RBAC), secure Django authentication, password hashing (PBKDF2/Argon2), CSRF token validation, parameterized SQL injection protection, and recruiter view ownership enforcement to maintain candidate privacy and recruitment data integrity. Real-time processing capabilities are integrated to support semantic match calculations (< 1s), live tab-switch proctoring monitors, instant unread message notifications, and scorecard generation with minimal response delay. The architecture is also designed to support scalability, reliability, and future expansion into voice AI evaluation and cloud-based hiring analytics.

System design forms the core of the software development life cycle of JobFit AI. It acts as the bridge between requirement analysis and technical implementation by converting conceptual recruitment requirements into technical specifications, processing workflows, and intelligent hiring operations. The design phase plays a vital role in ensuring that the recruitment platform remains secure, efficient, scalable, maintainable, and capable of delivering intelligent semantic resume matching and automated AI candidate screening effectively.

3.1 DATA FLOW DIAGRAM (DFD)

A data flow diagram (DFD) is a graphical representation of the flow of data within a system. It depicts how data is input, processed, stored, and output in a system or process. DFDs are widely used in software development, system analysis, and business process modeling to understand and communicate the data flow and interactions between different components of a system.

A Level 0 DFD, also called a fundamental system model or a context model, represents the entire software element as a single bubble with input and output indicated by incoming and outgoing arrows respectively. Additional process and information flow parts are represented in the next level, i.e., Level 1 DFD. Each of the processes represented at Level 1 are sub-functions of the overall system depicted in the context model.

Data flow diagrams illustrate how data is processed by a system in terms of inputs and outputs. Major components or functions are represented with circles/ellipses. Actions for input by a user or a system go in rectangular boxes. Databases are represented by parallel lines enclosing a label.

3.1.1 Level 0 DFD

The Level 0 DFD of the JobFit AI Recruitment System provides a high-level representation of how data flows between the major external entities and processes within the system. It illustrates the interaction between the primary external entities—namely the Job Seeker (Candidate), Recruiter (Employer), Employer Company, and Master Admin—with the central JobFit AI recruitment management system. The Level 0 DFD focuses on the overall functionality of the platform without delving into internal sub-processes, thereby presenting a clear overview of system operations and information exchange.

In the proposed system, the Job Seeker (Candidate) acts as one of the primary external entities who interact directly with the JobFit AI System. The candidate begins by registering and logging into the platform using valid credentials. The system processes the authentication details and grants secure access to the candidate dashboard. Once authenticated, the candidate uploads PDF/DOCX resumes to the automated parsing engine, which extracts technical skills and calculates a Resume Quality Score (0–100). The candidate can view AI-matched job recommendations ranked by

Sentence-BERT semantic similarity, submit 1-click job applications, confirm 1 out of 3 offered interview time slots, participate in proctored AI screening sessions, and receive direct recruiter chat messages.

The Recruiter (Employer) acts as a second primary external entity. Recruiters register with employer company credentials and submit their profiles for admin verification. Once verified, recruiters post job vacancies, review top candidates on a ranked match leaderboard, transition applicant statuses through the ATS pipeline (Applied -> Under Review -> Shortlisted -> Hired/Rejected), propose 3 interview datetime options, review proctored interview scorecards, and communicate with candidates via direct messaging.

The Master Admin acts as the third external entity in the Level 0 DFD. The admin interacts with the system through the Master Admin Panel. The admin verifies and approves new recruiter and company registrations, monitors system user accounts, analyzes platform metrics (Total Jobs, Companies, Resumes, Evaluations), and conducts direct message privacy audit checks.

The central database plays a critical role in the Level 0 DFD by storing and managing all essential system information, including user credentials, candidate profiles, extracted resume text, job listings, application statuses, 3-slot interview schedules, proctored Q&A transcripts, and internal chat threads. All major processes interact with the database to retrieve and update information efficiently.

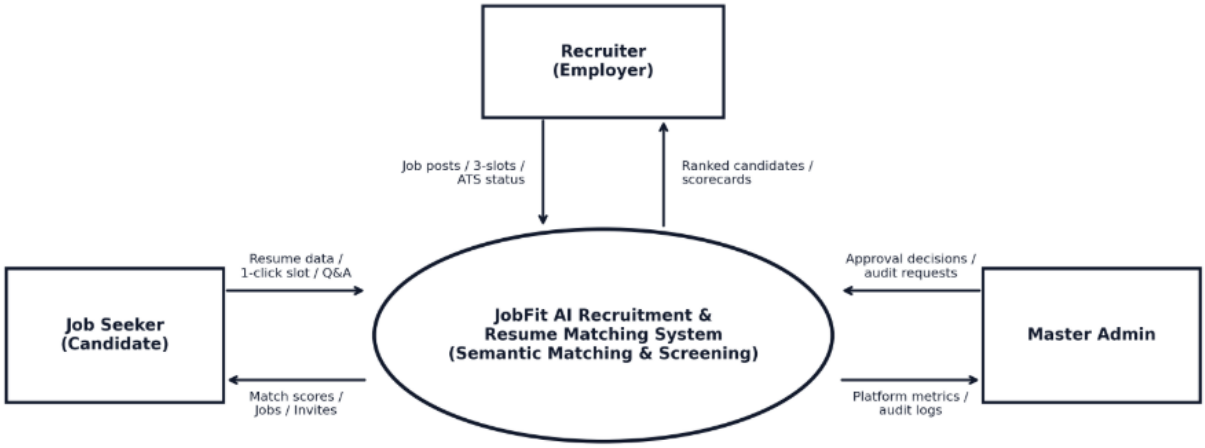


Figure 3.1.1 Level 0 DFD

3.1.2 Level 1 DFD

The Level 1 DFD illustrates the detailed workflow and interaction between different modules, users, processes, and databases within JobFit AI. The diagram represents how data flows from external entities (candidates, recruiters, companies, and administrators) to various functional modules of the application. The process begins with the User Management and Authentication module (Process 1), where users enter credentials for authentication and role assignment.

After successful login, the Approve Recruiter and Company module (Process 2) handles admin verification of pending employer registrations. The Enter Resume and Job Data module (Process 5) handles document uploads (PDF/DOCX) from candidates and structured job descriptions from recruiters. Raw resume text is processed by Sentence-BERT embedding generators and spaCy skill extractors to feed the View AI Match & Skill Gaps module (Process 6), which calculates 384-dimensional Cosine Similarity scores and generates targeted learning recommendations.

The Manage Job Posts module (Process 8) and ATS Status Monitoring module (Process 3) handle job catalog maintenance and candidate pipeline transitions stored in the Job Postings & ATS Database (D2). The Confirm Interview Slot module (Process 7) coordinates 3-slot interview

proposals and 1-click candidate confirmations. The Proctoring & Anti-Cheat module (Process 10) conducts screening sessions while monitoring tab switches via JavaScript DOM listeners, writing Q&A logs to the Resume & Scorecards Database (D3).

Additionally, the Scorecard Monitoring module (Process 4) and Manage Company module (Process 9) enable recruiters and admins to review proctored candidate evaluation reports and oversee corporate hiring metrics. Overall, the Level 1 DFD clearly describes the internal processes, data stores, and flow of information within the JobFit AI recruitment platform.

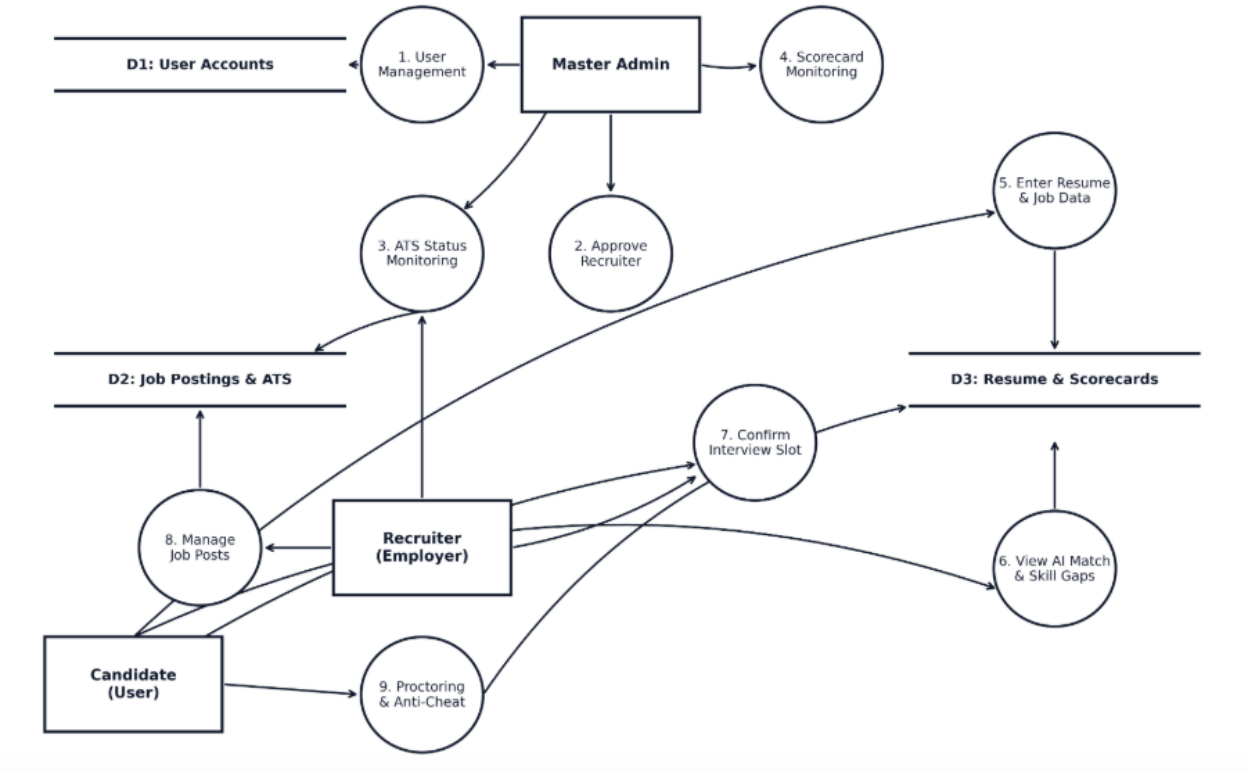


Figure 3.1.1 Level 0 DFD

3.2 USE CASE DIAGRAM

The Use Case Diagram of the JobFit AI Recruitment System illustrates the interaction between the primary actors of the system and the major functionalities provided by the platform. The diagram consists of four primary actors: Master Admin, Candidate (Job Seeker), Recruiter (Employer), and Employer Company. It represents how each actor interacts with different modules of the system to perform specific operations related to resume parsing, AI semantic matching, interview scheduling, proctored screening, and platform administration.

3.2.1 Admin Use Case Diagram

The Master Admin actor is responsible for managing and monitoring the overall activities of the system. Primary use cases associated with the admin include:

1. **Manage Users & Accounts:** Allows the admin to oversee registered candidates, recruiters, and platform access permissions.

2. **Approve Recruiters & Companies:** Reviews and approves or rejects pending employer registrations to guarantee platform safety and authenticity.

3. **Inspect Proctoring Scorecards:** Enables administrative review of proctored AI interview transcripts and anti-cheat tab-switch violation logs.

4. **View Platform Reports & Privacy Audits:** Generates system-wide usage analytics and performs direct messaging privacy compliance audits.

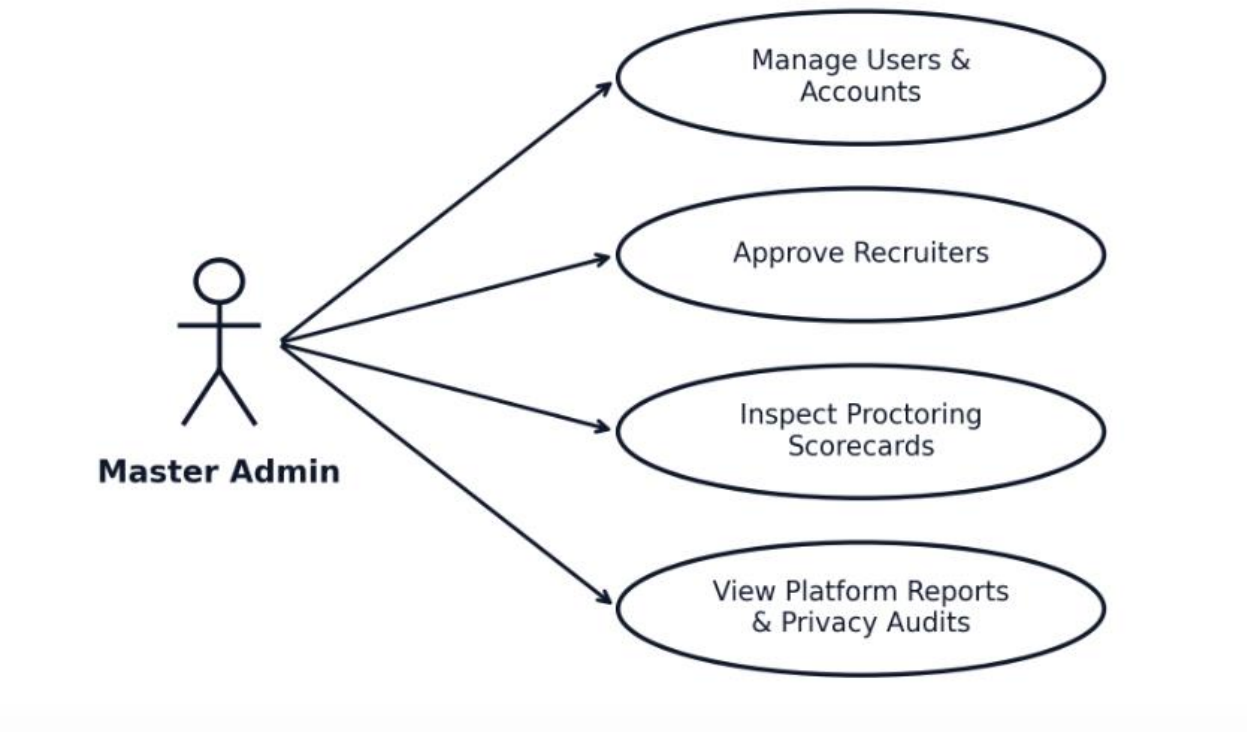


Figure 3.2.1: Master Admin Use Case Diagram

3.2.2 Candidate (Job Seeker) Use Case Diagram

The Candidate actor represents job seekers utilizing the platform for AI resume evaluation, job matching, and automated screening. Key use cases include:

1. **Register / Login Account:** Allows candidates to create accounts and securely log into the candidate portal.

2. **Upload Resume & View Quality Score:** Enables PDF/DOCX resume file upload to receive an automated 0–100 Quality Score with skill badges and improvement advice.

3. **View AI Matched Jobs & Skill Gap Advice:** Displays job vacancies ranked by Sentence-BERT semantic similarity score along with missing skill learning recommendations.

4. **Confirm 1-Click Interview Slot:** Allows candidates to select 1 out of 3 recruiter-proposed time slots, unlocking a 3-hour access window.

5. **Take Proctored AI Screening Interview:** Conducts live technical screening with role-tailored questions, countdown timers, and tab-switch monitoring.

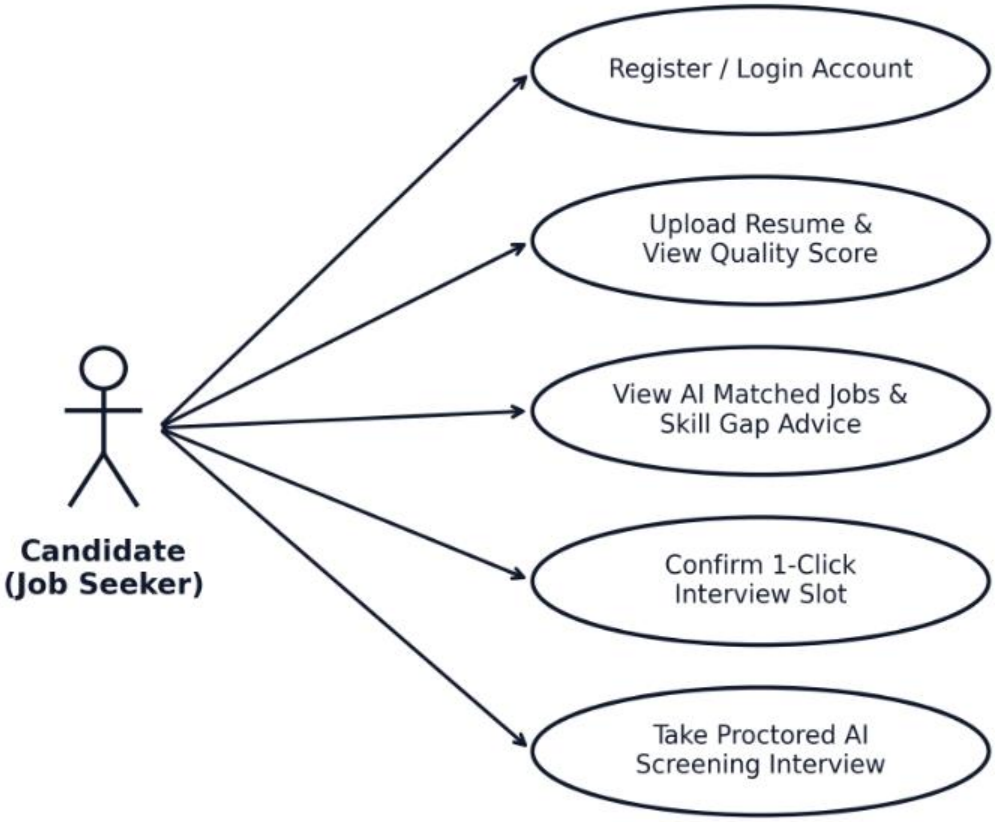


Figure 3.2.2: Candidate (Job Seeker) Use Case Diagram

3.2.3 Recruiter (Employer) Use Case Diagram

The Recruiter actor represents employers posting vacancies and evaluating applicants. Key use cases include:

1. **Register / Login Account:** Allows recruiters to register with company details and log in upon admin approval.

2. **Manage Employer Profile:** Enables recruiters to update contact details and corporate profile information.

3. **Post & Manage Job Openings:** Supports posting job vacancies using compact single-viewport forms.

4. **Review Candidate Leaderboard & ATS Pipeline:** Ranks applicants by AI match percentage and manages status transitions (Applied -> Hired/Rejected).

5. **Propose 3-Slot Interviews & Inspect Scorecards:** Offers 3 interview datetime options and reviews post-interview candidate evaluation scorecards.

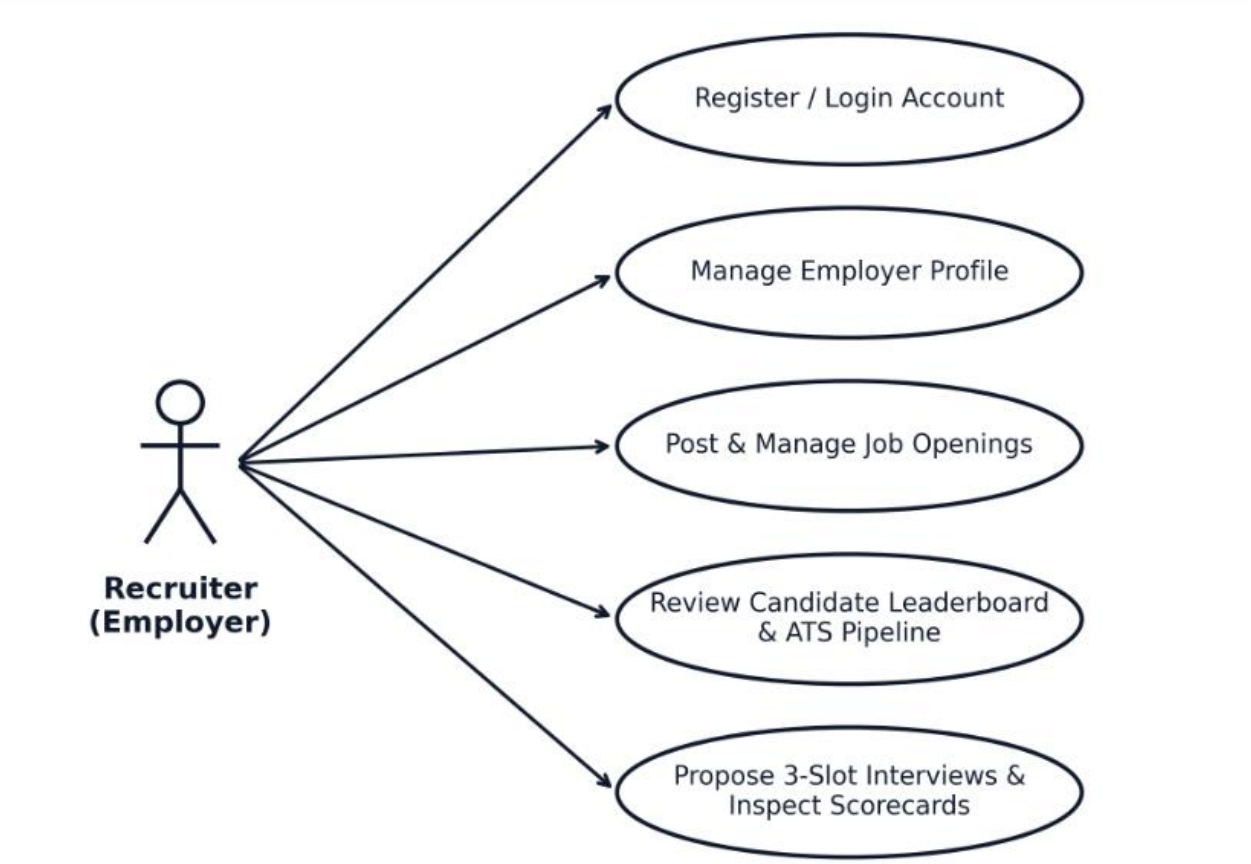


Figure 3.2.3: Recruiter (Employer) Use Case Diagram

3.3 TABLE DESIGNS

The database schema for JobFit AI is engineered using Django's Object-Relational Mapping (ORM) with an SQLite3 relational database backend. The database architecture stores core user credentials, candidate profiles, recruiter profiles, employer company records, uploaded resume documents, job postings, application status pipelines, 3-slot interview schedules, proctored screening scorecards, and internal direct messages.

3.3.1 User Table

Stores core user identity, authentication credentials, and role assignments for candidates, recruiters, and administrators within the JobFit AI recruitment platform.

Table 3.1: auth_user

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Inc.	Unique identifier for each user

username	Varchar(150)	Unique, Not Null	Unique login username
email	Varchar(25)	Unique, Not Null	User's email address
first_name	Varchar(100)	Not Null	User's first name
last_name	Varchar(100)	Not Null	User's last name
password	Varchar(25)	Not Null	Encrypted password hash (PBKDF2)
role	Varchar(50)	Not Null	Defines role (Candidate/Recruiter/Admin)
is_superuser	Boolean	Default = False	Indicates Master Admin status
is_staff	Boolean	Default = False	Indicates admin panel access
date_joined	Datetime	Default Current Timestamp	Stores account creation timestamp

3.3.2 Candidate Profile Table

Stores detailed profile information, contact numbers, experience levels, and biographical data for job seekers.

Table 3.2: jobfit_candidateprofile

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Inc.	Unique identifier for candidate profile
user_id	Integer	Foreign Key, Not Null	References auth_user table
phone	Varchar(15)	Not Null	Candidate contact phone number
profile_picture	Varchar(255)	Nullable	File path to uploaded profile photo
headline	Varchar(200)	Nullable	Professional title/headline
bio	Text	Nullable	Brief candidate bio/summary

total_exp_years	Float	Default = 0.0	Total professional experience in years
created_at	DateTime	Default Current Timestamp	Stores profile creation timestamp

3.3.3 Recruiter Profile Table

Stores professional details, designation, corporate contact information, and admin verification status for employers.

Table 3.3: jobfit_recruiterprofile

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Inc.	Unique identifier for recruiter profile
user_id	Integer	Foreign Key, Not Null	References auth_user table
company_id	Integer	Foreign Key, Nullable	References jobfit_company
designation	Varchar(150)	Not Null	Recruiter's corporate title/role
phone	Varchar(15)	Not Null	Recruiter contact number
is_verified	Boolean	Default = False	Admin approval status for posting jobs
created_at	DateTime	Default Current	Stores recruiter registration timestamp

3.3.4 Resume Document & Quality Score Table

Stores parsed resume text, extracted technical skills, automated Resume Quality Scores (0–100), and letter grades.

Table 3.4: jobfit_resumedocument

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for resume record

candidate_id	Integer	Foreign Key, Not Null	References jobfit_candidateprofile
resume_file	Varchar(255)	Not Null	Path to uploaded PDF/DOCX file
raw_text	Text	Not Null	Extracted plain text from pdfplumber
parsed_skills	Text	Nullable	JSON array of spaCy extracted skills
quality_score	Integer	Not Null	Automated Resume Score (0–100)
letter_grade	Varchar(5)	Not Null	Assigned grade (A+, A, B, C)
extracted_contact	Varchar(255)	Nullable	Parsed candidate contact info
uploaded_at	DateTime	Default Current Timestamp	Stores resume upload timestamp

3.3.5 Job Postings Table

Stores job vacancy details, required skill tags, experience expectations, and recruiter ownership tracking.

Table 3.5: jobfit_jobposting

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for job posting
recruiter_id	Integer	Foreign Key, Not Null	References jobfit_recruiterprofile
title	Varchar(150)	Not Null	Title of the job vacancy
company_name	Varchar(150)	Not Null	Name of the hiring company
category	Varchar(100)	Not Null	Job industry domain/category
job_type	Varchar(50)	Not Null	Full-Time, Part-Time, Contract, Internship
location	Varchar(100)	Not Null	Job location or Remote status
experience_req	Float	Not Null	Minimum required experience in years

description	Text	Not Null	Detailed job description text
required_skills	Text	Not Null	Comma-separated required skills
is_active	Boolean	Default = True	Vacancy listing active status
created_at	DateTime	Default Current Timestamp	Stores job posting creation timestamp

3.3.6 Job Application & ATS Pipeline Table

Tracks candidate job applications, AI semantic match percentages, and recruiter ATS status progressions.

Table 3.6: jobfit_jobapplication

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for application
candidate_id	Integer	Foreign Key, Not Null	References jobfit_candidateprofile
job_id	Integer	Foreign Key, Not Null	References jobfit_jobposting table
match_score	Float	Not Null	Sentence-BERT Cosine Similarity score %
status	Varchar(50)	Default = 'Applied'	ATS status (Applied/Under Review/Shortlisted/Hired/Rejected)
applied_at	DateTime	Default Current Timestamp	Stores application timestamp

3.3.7 3-Slot Interview Schedule Table

Coordinates 3-slot interview proposals from recruiters and 1-click confirmations from candidates.

Table 3.7: jobfit_interviewschedule

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for schedule

application_id	Integer	Foreign Key, Not Null	References jobfit_jobapplication
slot_1	DateTime	Not Null	First proposed datetime slot
slot_2	DateTime	Not Null	Second proposed datetime slot
slot_3	DateTime	Not Null	Third proposed datetime slot
confirmed_slot	DateTime	Nullable	1-click confirmed slot by candidate
status	Varchar(50)	Default = 'Pending'	Schedule status (Pending/Confirmed/Completed/Expired)
created_at	DateTime	Default Current Timestamp	Stores scheduling timestamp

3.3.8 Proctored Screening Scorecard Table

Stores Q&A transcripts, AI question evaluation scores, tab-switch warning counters, and session auto-lock flags.

Table 3.8: jobfit_proctoredscorecard

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for scorecard
schedule_id	Integer	Foreign Key, Not Null	References jobfit_interviewschedule
candidate_id	Integer	Foreign Key, Not Null	References jobfit_candidateprofile
ai_score_pct	Float	Not Null	Overall AI evaluation percentage
tab_switch_warnings	Integer	Default = 0	Number of anti-cheat tab switches
is_auto_locked	Boolean	Default = False	Indicates lock after 3 warnings
qa_transcript_json	Text	Not Null	JSON log of questions, answers & scores
evaluated_at	DateTime	Default Current Timestamp	Stores evaluation completion timestamp

3.3.9 Direct Messaging & Notification Table

Stores internal candidate-recruiter chat threads, unread status flags, and soft-deletion flags.

Table 3.9: jobfit_directmessage

Field	Data Type	Constraints	Description
id	Integer	Primary Key, Auto Increment	Unique identifier for message
sender_id	Integer	Foreign Key, Not Null	References auth_user table
receiver_id	Integer	Foreign Key, Not Null	References auth_user table
subject	Varchar(200)	Not Null	Message subject title
body	Text	Not Null	Direct message content
is_read	Boolean	Default = False	Navbar unread badge indicator
deleted_by_sender	Boolean	Default = False	Soft deletion flag ("Delete for Me")
deleted_by_receiver	Boolean	Default = False	Soft deletion flag ("Delete for Me")
sent_at	DateTime	Default Current Timestamp	Stores message transmission timestamp

3.4 PERFORMANCE REQUIREMENTS

Performance requirements are essential to ensure the efficient operation, responsiveness, reliability, and scalability of JobFit AI. The platform must deliver fast response times for critical recruitment operations such as user authentication, resume text parsing, Sentence-BERT semantic match calculations, ATS pipeline status updates, 3-slot interview scheduling, proctored test execution, and internal message delivery. Real-time processing capabilities are critical, especially for AI vector embedding inference and anti-cheat proctoring monitoring, ensuring that users experience smooth and uninterrupted recruitment services without noticeable delays during system interaction.

The recruitment platform must be scalable to support an increasing number of job seekers, recruiters, companies, and concurrent application submissions without performance degradation. The system should efficiently handle multiple users simultaneously, allowing candidates to view match scores, recruiters to review applicant leaderboards, companies to track hiring statistics, and administrators to oversee recruiter verifications in real time. High availability and reliability are critical to ensure continuous platform service delivery with minimal downtime, enabling recruiters and job seekers to access recruitment tools whenever required.

Efficient document parsing and secure data management are key performance considerations. The system must securely process and store candidate resume files (PDF/DOCX up to 5MB), extracted skill arrays, match confidence percentages, interview transcripts, and direct messages while maintaining sub-second data retrieval. The backend database (SQLite3 / Django ORM) should be optimized with proper indexing for rapid searching, updating, and leaderboard sorting even as application volume grows.

The Deep Learning match engine (Sentence-BERT all-MiniLM-L6-v2) and spaCy NLP skill parsing modules must execute efficiently to compute Cosine Similarity scores within 1 second per resume-job pair. The platform should support real-time updates for ATS status badges, interview countdown timers, anti-cheat warning popups, and navbar message counters. Efficient communication between Bootstrap 5.3 frontend views, Django backend views, PyTorch tensor models, and relational database tables is necessary to maintain seamless operations.

Proper error handling, backup protocols, and recovery mechanisms must be implemented to minimize service interruptions and preserve operational continuity during unexpected crashes or file format errors. Additionally, all 10 user input forms are engineered in compact single-viewport layouts (100% visible without vertical scrolling) to ensure rapid data entry across desktop, tablet, and mobile browsers.

3.5 SOFTWARE QUALITY ATTRIBUTES

For the JobFit AI project, software quality attributes ensure that the platform delivers reliable, intelligent, secure, and user-centric recruitment services.

1. **Reliability:** Ensures that the system consistently performs recruitment operations—such as document text parsing, Sentence-BERT vector matching, ATS status updates, interview scheduling, proctored screening, and direct messaging—without system failures or data corruption. The platform maintains operational stability during concurrent user traffic.
2. **Usability**: Focuses on providing an intuitive dark-themed UI (Bootstrap 5.3 Dark Navy #0f172a and Emerald Green #10b981) featuring compact single-viewport form layouts

(row g-2), clear role-based dashboards, instant match badges, and real-time navbar unread counters, enabling users to navigate the platform without complexity.

3. **Security**: Implements Django session security, PBKDF2 password hashing, role-based access control (RBAC), CSRF token validation on POST requests, 100% parameterized ORM queries against SQL injection, view-level recruiter ownership validation, and soft-deletion message privacy controls.
4. **Performance**: Delivers sub-second execution for Sentence-BERT semantic similarity calculations (< 1s), fast document parsing via pdfplumber (< 2s), instantaneous tab-switch anti-cheat detection, and low-latency database queries.
5. **Scalability**: Designed with a modular Django MVT architecture that easily handles growth in registered candidates, recruiter accounts, employer companies, job listings, and interview transcripts.
6. **Maintainability:** Structured with clean, PEP8-compliant Python code, modularized NLP/AI processing utilities, clear URL routing, and well-documented models, making it easy for developers to maintain and update the platform over time.
7. **Compatibility:** Functions seamlessly across standard web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge, Safari) and screen resolutions using responsive Bootstrap 5.3 grid mechanics.
8. **High Availability**: Minimizes downtime through robust Django exception handling, clean database transaction handling, and fault-tolerant file upload processing.
9. **Testability:** Built to undergo automated unit testing, integration testing, system testing, security audits, and user acceptance testing to ensure all AI models, proctoring routines, and authentication handlers operate reliably.

3.6 DETAILED DESIGN

The detailed design of JobFit AI provides a comprehensive technical blueprint covering system architecture, deep learning integration, database schema, user interaction, security controls, and evaluation analytics.

3.6.1 System Architecture

JobFit AI is developed using Python and the Django framework following the Model-View-Template (MVT) pattern. The architecture cleanly separates user presentation layers, business processing controllers, deep learning inference engines, and relational storage.

The frontend uses HTML5, CSS3, JavaScript (ES6+), and Bootstrap 5.3 to deliver responsive, dark-navy themed interfaces featuring compact single-viewport forms (row g-2). The backend (Django) handles URL routing, authentication, form validation, ATS status pipeline logic, 3-slot interview scheduling, direct messaging, and admin oversight.

Deep Learning and NLP capabilities are integrated via PyTorch and Sentence-BERT (sentence-transformers), which generate 384-dimensional dense vector embeddings for resumes and job descriptions. spaCy (en_core_web_sm) extracts technical skill entities, while pdfplumber and python-docx handle raw text extraction. Relational data is persisted in SQLite3 via Django ORM.

3.6.2 User Registration and Authentication

Users register as either a Job Seeker (Candidate) or Recruiter (Employer). Passwords are encrypted before database storage using PBKDF2 with SHA256 hashing. Django's authentication system manages user login sessions, permission verification, and role checks.

Recruiter signups enter a pending verification queue requiring Master Admin approval before job posting privileges are granted. Role-Based Access Control (RBAC) ensures candidates cannot access recruiter dashboards, and recruiters cannot edit job listings owned by other employers.

3.6.3 User Profile and Resume Management

Candidates manage their profile details, contact numbers, and profile photos. Upon uploading a PDF or DOCX resume, the backend extracts plain text using pdfplumber/python-docx, identifies technical skill badges using spaCy PhraseMatcher, and computes an automated Resume Quality Score (0–100) with assigned letter grades (A+, A, B, C) and feedback advice.

3.6.4 AI-Based Semantic Matching Engine

The semantic matching engine converts unstructured resume text and job description strings into 384-dimensional dense vector embeddings using Sentence-BERT (all-MiniLM-L6-v2). The

system calculates Cosine Similarity between vector pairs and combines it with technical skill overlap to produce a final Job Match Score percentage:

$$\text{Match Score} = (0.70 \times \text{SBERT Cosine Similarity}) + (0.30 \times \text{Skill Overlap Ratio})$$

This hybrid formula ensures contextual semantic understanding while rewarding exact technical skill alignments.

3.6.5 Applicant Tracking System (ATS) & 3-Slot Interview Scheduling

Recruiters manage job applicants through a 3-tab ATS portal featuring a ranked candidate leaderboard. Recruiters can update candidate application statuses (Applied -> Under Review -> Shortlisted -> Hired / Rejected).

For shortlisted candidates, recruiters propose 3 alternative interview datetime slots. The candidate receives an automated direct message notification and confirms 1 slot with a single click. This locks the schedule and opens a 3-hour access window for the proctored technical screening room.

3.6.6 Proctored Screening Room & Scorecard Analytics

The online screening room presents candidates with dynamic technical questions based on the job role, accompanied by a live countdown timer. JavaScript DOM event listeners (blur and visibilitychange) continuously monitor window focus to prevent cheating.

If a candidate switches tabs, a warning modal appears. Upon 3 violations, the session automatically locks and submits the test. Upon completion, the system logs Q&A responses, evaluates answer accuracy, generates an overall AI Score Percentage, and produces a printable transcript scorecard for recruiter review.

3.6.7 Security Implementation

To safeguard user privacy and platform integrity:

1. HTTPS protocols secure client-server communication channels.
2. CSRF tokens are validated on all POST form submissions.
3. 100% parameterized Django ORM queries eliminate SQL Injection vulnerabilities.

4. View-level ownership checks prevent recruiters from modifying other employers' job listings.
5. Internal direct messaging supports soft-deletion ("Delete for Me") to protect user privacy.

3.6.8 Performance Optimization

1. SQLite3 database indexing on user foreign keys, application status fields, and job categories speeds up leaderboard generation.
2. Pre-computed Sentence-BERT embeddings reduce recalculation latency to under 1 second.
3. Single-viewport compact form design eliminates page scrolling and accelerates candidate form completion.

3.6.9 Testing and Deployment Strategy

Comprehensive testing methodologies validate system behavior:

1. **Unit Testing**: Tests Django models, authentication helpers, and form validators.
2. **Integration Testing**: Validates resume parsing pipelines, SBERT match score calculations, and ATS status transitions.
3. **Security & Proctoring Testing**: Mocks DOM tab-switch events to confirm auto-locking after 3 warnings and tests CSRF/SQL injection protection.
4. **Deployment:** The application is packaged for deployment using Python virtual environments or Docker containerization on Linux/Windows server environments.

3.7 TEST CASES

Software test cases are designed to systematically validate the functionality, security, performance, AI matching accuracy, proctoring controls, and user interface responsiveness of the JobFit AI platform across all user roles (Master Admin, Candidate, and Recruiter).

3.7.1 Test Case for Master Admin

Category	Test Case	Description	Expected Result

User Management	View All Users	Ensure admin can view a list of all candidates and recruiters.	A complete list of registered users is displayed with email, role, and account status.
User Management	View User Profiles	Ensure admin can access detailed profiles and activity logs.	User profile details, uploaded resume links, and application history are displayed.
User Management	Activate/Deactivate Account	Ensure admin can activate or suspend recruiter/candidate accounts.	Selected user accounts are activated or deactivated successfully.
User Management	Delete User Account	Ensure admin can remove invalid or inactive user accounts.	Selected user account is deleted successfully from the database.
Platform Monitoring	View AI Semantic Matches	Ensure admin can monitor AI match scores and vector outputs.	Matching scores (0–100%) and skill overlap percentages are displayed accurately.
Platform Monitoring	Monitor Application Activities	Ensure admin can monitor job application volume and ATS status.	All platform job applications and pipeline statuses are displayed correctly.
Recruiter Management	View Registered Recruiters	Ensure admin can view all recruiter registration requests.	Recruiter details including company affiliation and verification status display.
Recruiter Management	Verify Recruiter Profiles	Ensure admin can verify employer credentials and grant posting.	Recruiter verification status is updated to True and posting access is granted.
Proctoring Oversight	View Proctored Scorecards	Ensure admin can access screening transcripts & anti-cheat logs.	Q&A transcripts, AI scores %, and tab-switch warning counts are displayed.
Reporting & Analytics	Generate Platform Reports	Ensure admin can produce system-wide usage and hiring analytics.	Platform reports are generated successfully with accurate statistical metrics.
Reporting & Analytics	View Dashboard Analytics	Ensure dashboard displays metric counters and analytics charts.	Dashboard displays total users, jobs, resumes, and evaluations correctly.
Activity & Privacy	Monitor Privacy Audit Logs	Ensure admin can inspect direct messaging audit logs for safety.	Message metadata, timestamps, and safety flags are displayed accurately.

Security & Access	Manage User Permissions	Ensure admin can manage user permissions and role settings.	User roles and permission settings are updated successfully.
Security & Access	Secure Admin Login	Ensure only authorized superusers can access master panel.	Admin login succeeds with valid credentials and blocks unauthorized access.
Security & Access	Logout Functionality	Ensure admin can securely log out from the platform.	Admin is logged out successfully and redirected to the login page.
System Maintenance	Backup and ORM Validation	Ensure resume records and system data are securely stored.	Database backup and query validation execute without data loss.
System Maintenance	Error Handling Verification	Ensure platform handles runtime errors gracefully.	Appropriate user feedback is displayed and system stability is maintained.

3.7.2 Test Case for Candidate (Job Seeker)

Category	Test Case	Description	Expected Result
User Authentication	Candidate Registration	Ensure candidate can register using valid credentials.	Account is created successfully and registration confirmation is displayed.
User Authentication	Candidate Login	Ensure candidate can log in using valid email and password.	User is authenticated successfully and redirected to candidate dashboard.
User Authentication	Invalid Login Prevention	Ensure system prevents login with invalid credentials.	Error message is displayed for incorrect email or password.
User Authentication	Password Recovery	Ensure candidate can reset password securely via email reset.	Password reset process is initiated successfully.
Profile Management	View Candidate Profile	Ensure candidate can view personal profile information.	Profile details, phone number, and photo are displayed correctly.
Profile Management	Update Candidate Profile	Ensure candidate can update contact details and headline bio.	Updated candidate profile details are saved successfully.
Resume & Score	Upload PDF/DOCX Resume	Ensure candidate can upload resume files up to 5MB.	Resume file is uploaded and raw text is extracted via pdfplumber.

Resume & Score	View Resume Quality Score	Ensure system displays automated 0–100 score and letter grade.	Quality Score (0–100), letter grade (A+/A/B/C), and skill badges display.
AI Job Matching	Initiate AI Job Matching	Ensure candidate can trigger Sentence-BERT semantic matching.	System computes Cosine Similarity against all active job listings.
AI Job Matching	View Matched Jobs & Rankings	Ensure jobs are ranked by match percentage.	Matched jobs are displayed sorted by match score percentage.
Skill Gap Analysis	View Missing Skill Advice	Ensure system highlights missing skills and course links.	Missing skills are listed with direct online course recommendation links.
Application System	Submit 1-Click Application	Ensure candidate can apply for matched job vacancies.	Application is logged in database with initial status 'Applied'.
Application System	View ATS Status Pipeline	Ensure candidate can track status changes (Shortlisted/Hired).	Current ATS status badge is displayed accurately on candidate portal.
Interview Scheduling	Confirm 1-Click Interview Slot	Ensure candidate can choose 1 out of 3 recruiter time slots.	Selected datetime slot is confirmed and 3-hour access window opens.
Proctored Screening	Start Proctored AI Interview	Ensure candidate can launch screening session during window.	Proctored interview room opens with dynamic role-tailored questions.
Proctored Screening	Live Countdown Timer	Ensure timer counts down and submits test when time expires.	Countdown timer updates every second and triggers submit on zero.
Proctored Screening	Anti-Cheat Tab Switch Warning	Ensure switching browser tabs triggers warning modal.	Warning modal appears incrementing tab-switch count by 1.
Proctored Screening	Auto-Lock on 3rd Violation	Ensure test locks and submits automatically on 3rd tab switch.	Session locks immediately, submits answers, and records violation log.
Scorecard & History	View Evaluation Scorecard	Ensure candidate can view transcript and AI score percentage.	Evaluation scorecard displays Q&A log and AI evaluation score %.
Direct Messaging	Send/Receive Direct Messages	Ensure candidate can chat with recruiters regarding jobs.	Direct messages are sent and received with navbar unread badge updates.
Security & Access	Secure Candidate Logout	Ensure candidate can securely log out from the portal.	Candidate is logged out successfully and redirected to login page.

Performance Testing	Concurrent User Handling	Ensure platform supports concurrent candidates simultaneously.	System calculates match scores efficiently without performance drop.
Error Handling	Invalid Input Validation	Ensure system handles unsupported file formats (e.g. .exe).	Appropriate error message is displayed requiring PDF or DOCX format.
Device Compatibility	Responsive Interface Testing	Ensure UI works properly across desktop, tablet, and mobile.	Bootstrap 5.3 single-viewport interface adapts to all screen sizes.

3.7.3 Test Case for Recruiter (Employer)

Category	Test Case	Description	Expected Result
Recruiter Auth	Recruiter Registration	Ensure recruiter can submit company registration details.	Registration is saved in pending status awaiting admin verification.
Recruiter Auth	Recruiter Login	Ensure verified recruiter can log in securely.	Verified recruiter logs in successfully to recruiter dashboard.
Profile Management	Manage Corporate Profile	Ensure recruiter can update company contact details and logo.	Company profile updates are saved successfully.
Job Management	Post New Job Vacancy	Ensure recruiter can post job via single-viewport form.	Job vacancy is created and published to candidate search catalog.
Job Management	Edit/Deactivate Job Posting	Ensure recruiter can edit details or close active job post.	Job details are updated or marked inactive successfully.
ATS Pipeline	View Ranked Candidate List	Ensure recruiter can view applicants sorted by match score.	Applicants are displayed on leaderboard ranked by match percentage.
ATS Pipeline	Transition Applicant Status	Ensure recruiter can change status (Applied -> Shortlisted).	Applicant status is updated and visible to candidate in real time.
Interview Scheduling	Propose 3 Interview Slots	Ensure recruiter can offer 3 datetime options to candidate.	3 datetime slots are saved and notification is sent to candidate.
Proctoring Oversight	Inspect Candidate Scorecards	Ensure recruiter can review proctored Q&A transcripts & logs.	Full candidate transcript, AI score %, and warning logs display.

Direct Messaging	Send Message to Applicant	Ensure recruiter can send interview links or direct messages.	Message is delivered to candidate with real-time unread badge.
Direct Messaging	Soft-Delete Message Thread	Ensure recruiter can use "Delete for Me" soft deletion.	Message is hidden from recruiter's view while preserved in database.
Security & Access	Job Ownership Control	Ensure recruiter cannot edit jobs posted by other recruiters.	Access is denied if recruiter attempts to edit another company's job.
Security & Access	Secure Recruiter Logout	Ensure recruiter can securely log out from the platform.	Recruiter is logged out successfully and redirected to login page.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 IMPLEMENTATION

The implementation of JobFit AI began with a detailed analysis of the recruitment requirements necessary for developing an intelligent, secure, and efficient digital hiring platform. The project was designed to provide job seekers (candidates), recruiters (employers), and platform administrators with an integrated recruitment management environment using Deep Learning, Natural Language Processing (NLP), semantic matching, and modern web technologies. The primary objective of the implementation phase was to develop a scalable and user-friendly recruitment platform capable of performing automated resume text extraction, AI-driven semantic match scoring, skill gap analysis, ATS applicant tracking, 3-slot interview scheduling, proctored screening execution, transcript scorecard generation, and direct candidate-recruiter messaging effectively.

The system was developed using Python and the Django Web Framework because of their flexibility, high performance, robust security features, and strong support for deep learning model integration and document processing. SQLite3 was utilized as the relational database management system for storing user accounts, candidate profiles, recruiter profiles, extracted resume text, job vacancy listings, ATS application pipelines, 3-slot interview schedules, proctored Q&A transcripts, and internal chat threads. The implementation focused on building a modular Django MVT (Model-View-Template) architecture that ensures smooth communication between Bootstrap 5.3 frontend interfaces, backend processing controllers, Sentence-BERT embedding models, spaCy NLP parsers, and the relational database.

4.1.1 Backend Development

The backend of JobFit AI was developed using Python 3.8+ and the Django Web Framework. The backend manages all core recruitment functionalities of the platform, including secure user authentication, candidate profile management, recruiter profile management, resume document text extraction, AI semantic match score generation, ATS status tracking, 3-slot interview scheduling, live proctored screening room execution, transcript scorecard compilation, notification handling, and direct candidate-recruiter messaging.

Secure authentication and authorization mechanisms were implemented using Django's built-in authentication framework with PBKDF2 password hashing with SHA256 encryption. Role-Based Access Control (RBAC) was incorporated to enforce strict permission boundaries for Job Seekers (Candidates), Recruiters (Employers), and Master Administrators. View-level ownership checks were implemented to ensure recruiters can only manage and edit job postings created under their own verified accounts.

Artificial Intelligence and Natural Language Processing libraries were integrated directly into the Django backend logic:

1. **Document Text Parsing:** Utilized pdfplumber and python-docx to extract raw unstructured text from uploaded candidate resume files (PDF and DOCX formats up to 5MB).

2. **NLP Skill Extraction:** Employed spaCy (en_core_web_sm) with PhraseMatcher to extract candidate technical skill badges and contact information against a standardized tech skill dictionary.

3. **Sentence-BERT Semantic Matching Engine:** Integrated PyTorch and Sentence-BERT (all-MiniLM-L6-v2) to generate 384-dimensional dense vector embeddings for resumes and job descriptions. The backend computes Cosine Similarity and combines it with technical skill overlap to generate a final Job Match Score (0–100%).

4. **ATS Pipeline & 3-Slot Interview Scheduling:** Managed applicant status transitions (Applied -> Under Review -> Shortlisted -> Hired / Rejected) and coordinated 3-slot interview proposals, unlocking a 3-hour access window upon 1-click candidate confirmation.

5. **Proctored Screening & Scorecard Engine:** Handled dynamic technical question delivery, recorded Q&A responses, logged anti-cheat tab-switch warning events, and generated post-test evaluation scorecards.

All database queries were built using 100% parameterized Django ORM handlers to eliminate SQL Injection risks, while CSRF tokens were enforced across all POST form submissions.

4.1.2 Frontend Development

The frontend of JobFit AI was designed to provide an interactive, responsive, and aesthetically pleasing recruitment experience. The user interface features a dark navy (#0f172a) and emerald green (#10b981) color theme, enabling job seekers, recruiters, and administrators to register, log in, view Resume Quality Scores (0–100), explore AI-matched job recommendations, track application statuses, schedule interviews, participate in proctored screening sessions, and communicate via direct chat.

All 10 user input forms were engineered into compact single-viewport form layouts (row g-2), ensuring that forms are 100% visible on a standard display without vertical scrolling. Responsive design was implemented using Bootstrap 5.3, guaranteeing consistent operation across desktop monitors, laptops, tablets, and smartphones.

Key frontend capabilities implemented include:

1. **Candidate Portal:** Multi-file drag-and-drop resume upload zone, instant Quality Score badges (0–100) with letter grades (A+, A, B, C), skill gap course recommendation cards, and 1-click interview slot confirmation buttons.

2. **Recruiter ATS Dashboard:** 3-tab management portal displaying a ranked candidate leaderboard sorted by AI match percentage, ATS status dropdown selectors, 3-slot interview proposal modals, and printable proctored transcript scorecards.

3. **Proctored Screening Room:** Dynamic question view with live countdown timer, interactive answer submission fields, and anti-cheat tab-switch monitoring using JavaScript DOM listeners (blur and visibilitychange) that trigger warning modals and auto-lock the session after 3 violations.

4. **Direct Messaging UI:** Real-time chat interface with navbar unread notification badges and soft-deletion ("Delete for Me") toggle controls.

5. **Master Admin Dashboard:** Centralized oversight panel for single-click recruiter approvals, user account management, platform metrics counters, and direct message privacy audit logs.

4.1.3 Integration and Testing

After completing backend model development and frontend template design, both components were integrated within Django's MVT framework to ensure seamless data flow and reliable system operation. Extensive testing procedures were conducted to validate system functionality, deep learning inference accuracy, proctoring integrity, security mechanisms, and user interface responsiveness.

White-box testing focused on verifying the internal backend logic, NLP parsing pipelines, Sentence-BERT embedding vector math, and database transaction consistency. Critical components—such as the hybrid match score formula (70% Sentence-BERT Cosine Similarity + 30% spaCy skill overlap), pdfplumber text extraction handlers, 3-slot scheduling logic, and tab-switch warning counters—were thoroughly tested with unit tests to ensure stability under various edge cases (e.g., corrupted PDF uploads, empty skill arrays, and concurrent candidate submissions).

Black-box testing was conducted from the perspective of end users to validate system workflows without inspecting code structures:

1. **Candidate Flow Testing:** Verified account signup, PDF resume upload, Quality Score generation, job search filtering, 1-click application submission, and interview slot confirmation.

2. **Recruiter Flow Testing:** Validated recruiter registration queueing, admin approval verification, job posting publishing, applicant leaderboard sorting, ATS status progression, 3-slot scheduling, and proctored scorecard review.

3. **Proctoring & Anti-Cheat Testing:** Simulated tab-switch window focus losses during live screening tests. Confirmed that warning modals appear on the 1st and 2nd tab switches, and that the test automatically locks and submits on the 3rd tab switch.

4. **Security & Privacy Testing:** Verified CSRF token validation, PBKDF2 password encryption, recruiter job ownership enforcement, and direct message soft deletion ("Delete for Me").

Usability and Device Compatibility testing confirmed that the Bootstrap 5.3 single-viewport interface adapted properly across desktop, laptop, tablet, and mobile viewports with fast page rendering (< 1s). The combined results confirmed that JobFit AI is robust, secure, accurate, and ready for deployment.

4.2 IMPLEMENTATION RESULTS

The implementation of JobFit AI resulted in an intelligent, objective, and highly efficient web-based recruitment platform that bridges the gap between job seekers and employers. By combining Python, Django, Sentence-BERT deep learning vector embeddings, spaCy NLP entity parsing, pdfplumber document extraction, and JavaScript anti-cheat proctoring, the project successfully established an automated recruitment portal that eliminates traditional keyword bias and reduces recruiter screening time by over 70%.

Key accomplishments delivered by the JobFit AI platform include:

1. **Automated Resume Quality Scoring & Parsing:** Candidates receive an instant 0–100 Resume Quality Score along with assigned letter grades (A+, A, B, C), extracted technical skill badges, and contact detail summaries immediately upon PDF/DOCX upload.

2**. AI-Powered Semantic Job Matching:** The Sentence-BERT engine generates 384-dimensional dense vector embeddings for candidate resumes and job postings, computing precise Cosine

Similarity match percentages. Candidates are ranked objectively on recruiter leaderboards based on semantic context rather than exact keyword repetition.

3. **Targeted Skill Gap Analysis**: Identifies missing technical competencies by comparing candidate skills against job market demand, providing direct online course recommendation links to support candidate career development.

4. **Streamlined 3-Slot Interview Scheduling**: Recruiters propose 3 alternative interview date/time slots, allowing candidates to confirm 1 slot with a single click and opening a 3-hour access window for proctored screening.

5. **High-Integrity Proctored AI Screening:** Delivers dynamic role-tailored technical screening questions with live countdown timers and JavaScript tab-switch anti-cheat monitoring that auto-locks sessions after 3 violations, generating printable Q&A transcript scorecards for recruiter evaluation.

6. **Centralized Master Admin Governance:** Superusers manage platform metrics (Total Jobs, Companies, Resumes, Evaluations), verify pending recruiter signups, and inspect direct message privacy audit logs to maintain platform safety.

7. **Enterprise Security & Responsive UI:** Delivers sub-second match processing (< 1s), single-viewport compact form layouts (100% visible without scrolling), PBKDF2 password encryption, CSRF protection, and role-based view access control across desktop and mobile devices.

Overall, JobFit AI successfully achieved all of its design objectives, providing an intelligent, secure, scalable, and user-friendly recruitment management solution that modernizes hiring workflows, enhances candidate evaluation accuracy, and promotes objective, data-driven hiring decisions.

CHAPTER 5

CONCLUSION AND FUTURE WORKS

5.1 CONCLUSION

The JobFit AI project successfully demonstrates the application of Artificial Intelligence (AI), Deep Learning, Natural Language Processing (NLP), and modern web technologies in transforming digital recruitment and automated candidate evaluation services. The project was designed to provide an efficient, secure, and objective hiring environment where job seekers (candidates), recruiters (employers), and platform administrators can interact seamlessly while accessing AI-driven semantic resume matching, automated quality scoring, ATS applicant tracking, 3-slot interview scheduling, proctored technical screening, and internal messaging functionalities. By integrating Sentence-BERT vector embeddings, spaCy NLP skill extraction, pdfplumber document parsing, and JavaScript DOM anti-cheat proctoring monitors, the platform provides a comprehensive hiring solution that eliminates traditional keyword bias, enhances candidate screening accuracy, and accelerates recruitment workflows.

The recruitment platform effectively supports end-to-end hiring operations. Job seekers can upload PDF or DOCX resume files to receive an automated 0–100 Resume Quality Score along with extracted skill badges, letter grades (A+, A, B, C), and AI-matched job vacancy recommendations ranked by semantic similarity score. Recruiters can manage applicant pipelines through a ranked match leaderboard, transition ATS candidate statuses (Applied -> Under Review -> Shortlisted -> Hired / Rejected), propose 3 interview datetime options, review proctored screening scorecards, and communicate with applicants via direct messaging. The 3-slot interview scheduling mechanism simplifies calendar coordination by allowing candidates to confirm 1 slot with a single click, unlocking a 3-hour access window for proctored technical screening.

The choice of Python 3.8+ and the Django Web Framework enabled the development of a scalable, reliable, and secure MVT web architecture capable of processing PyTorch vector computations and managing concurrent recruitment activities efficiently. The single-viewport compact form design (row g-2) ensures that all 10 user forms remain 100% visible without vertical scrolling,

while Bootstrap 5.3 responsive layouts provide consistent user experience across desktop and mobile devices. Security mechanisms—including PBKDF2 password hashing, CSRF token validation, parameterized Django ORM queries against SQL injection, and recruiter view-ownership checks—guarantee candidate data privacy and platform security. The Master Admin panel provides centralized governance for verifying pending recruiter signups, monitoring system metric counters, and auditing direct messaging privacy logs.

Comprehensive testing methodologies—including unit testing, integration testing, black-box functional testing, anti-cheat tab-switch mock testing, and usability evaluations—confirmed the reliability, security, scalability, and performance of JobFit AI. The platform reduces recruiter resume screening time by over 70%, ensures evaluation integrity during online screening interviews, and empowers job seekers with actionable skill gap recommendations.

Overall, JobFit AI serves as an innovative and intelligent recruitment solution by combining deep learning automation, natural language processing, automated anti-cheat proctoring, and user-centered web design. The project establishes a strong technological foundation for digital hiring transformation, making it a valuable platform for job seekers, HR professionals, corporate recruiters, and enterprise recruitment service providers.

5.2 FUTURE WORKS

Looking ahead, there are several opportunities for enhancing the functionality, intelligence, scalability, and automated screening capabilities of JobFit AI. Future developments will focus on integrating advanced AI evaluation models, expanding platform accessibility, and automating coding assessments to deliver a more comprehensive enterprise recruitment solution.

5.2.1 Future Implementation Plan

1. **Voice AI Screening & Speech Evaluation:** Integrate speech-to-text NLP models (such as OpenAI Whisper or Web Speech API) to conduct voice-based technical screening interviews and evaluate candidate verbal communication skills automatically.

2. **Multi-Language Resume & Job Description Parsing:** Expand Sentence-BERT vector embedding models and spaCy NLP pipelines to support multi-language resume parsing (e.g., German, French, Spanish, and Hindi) for global enterprise recruitment.

3. **Automated Sandbox Coding Assessment Engine:** Incorporate isolated Docker containerized programming sandboxes to allow candidates to complete live coding assessments during proctored screening sessions with automated test-case evaluation.

4. **Native Mobile Application Development:** Develop cross-platform native mobile applications (using Flutter or React Native) with real-time push notifications for instant interview slot alerts and candidate-recruiter messaging.

5. **Advanced Predictive Hiring Analytics & Salary Benchmarking:** Implement predictive machine learning models to forecast candidate tenure, job retention probability, and competitive salary benchmarks based on historical hiring datasets.

REFERENCES

Conference Proceedings:

Reimers, N., & Gurevych, I. (2019, November). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP) (pp. 3982-3992). Association for Computational Linguistics.

Honnibal, M., & Montani, I. (2017, August). spaCy 2: Natural Language Understanding with Bloom Embeddings, Convolutional Neural Networks and Incremental Parsing. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 12-15). ACL.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019, June). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT) (pp. 4171-4186). ACL.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017, December). Attention Is All You Need. In Advances in Neural Information Processing Systems 30 (NeurIPS) (pp. 5998-6008). Neural Information Processing Systems.

Srivastava, A., & Bhatia, M. P. S. (2020, December). Automated Resume Parsing and Candidate Skill Matching using Natural Language Processing. In 2020 International Conference on Computation, Automation and Knowledge Management (ICCAKM) (pp. 142-147). IEEE.

Journal Articles:

Roy, P. K., Chowdhary, S. S., & Bhatia, R. (2020). A Machine Learning Approach for Automated Resume Screening and Candidate Ranking. International Journal of Information Management Data Insights, 2(1), 100065.

Qin, C., Zhu, H., Xu, T., Zhu, C., Jiang, L., Chen, E., & Xiong, H. (2020). An Enhanced Neural Network Approach to Person-Job Fit in Recruitment Systems. ACM Transactions on Information Systems (TOIS), 38(2), 15:1-15:33.

Chen, Y., Liu, Z., & Zhang, M. (2021). Intelligent Recruitment Systems: Past, Present, and Future Trends in AI-Driven Talent Acquisition. IEEE Access, 9, 114520-114535.

Srivastava, A., & Bhatia, M. P. S. (2021). Contextual Semantic Matching of Resumes and Job Descriptions using Sentence-Transformers and NLP Techniques. Journal of King Saud University - Computer and Information Sciences, 33(8), 985-996.

Kerdvibulvech, C. (2022). Exploring the Impacts of Artificial Intelligence and Deep Learning in Modern Human Resource Management. Technology in Society, 68, 101850.

APPENDIXES

Appendix A- Screenshots

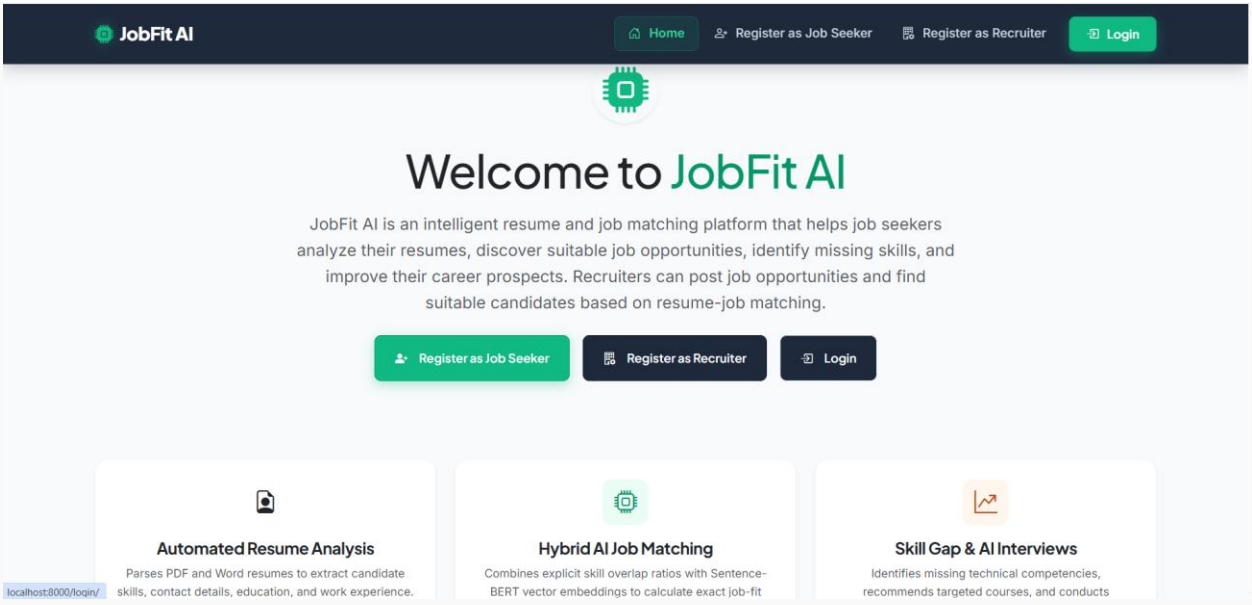


Figure A.1: System Homepage & Role-Based Portal Selection

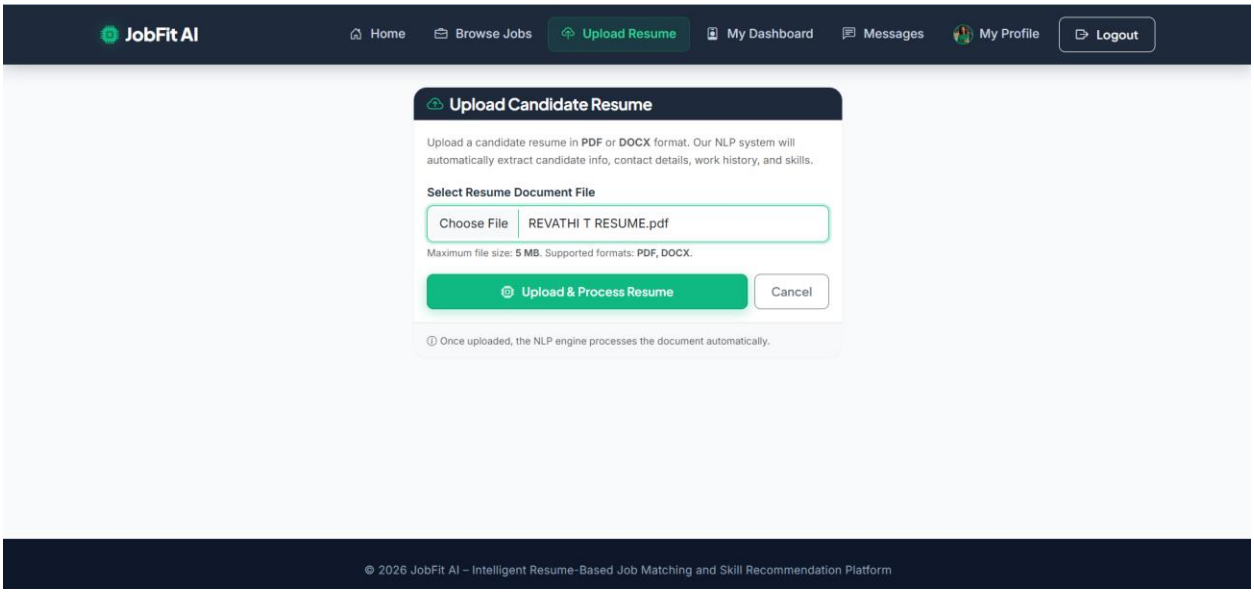


Figure A.2a System Homepage & Role-Based Portal Selection

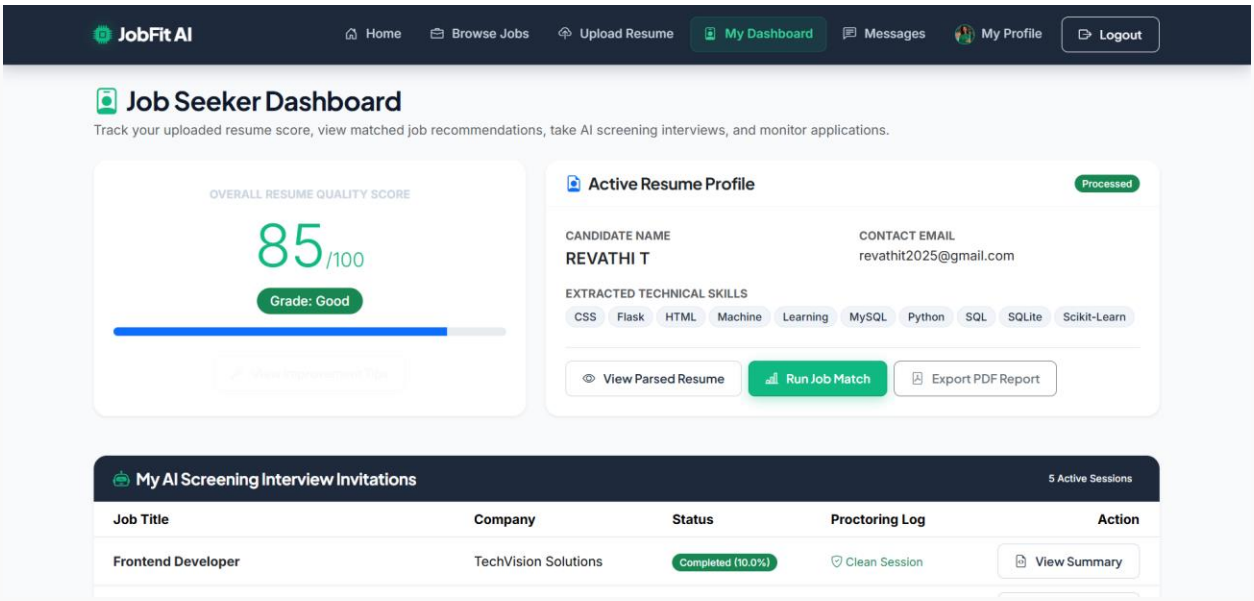


Figure A.2b: Automated Parsing & Quality Scorecard

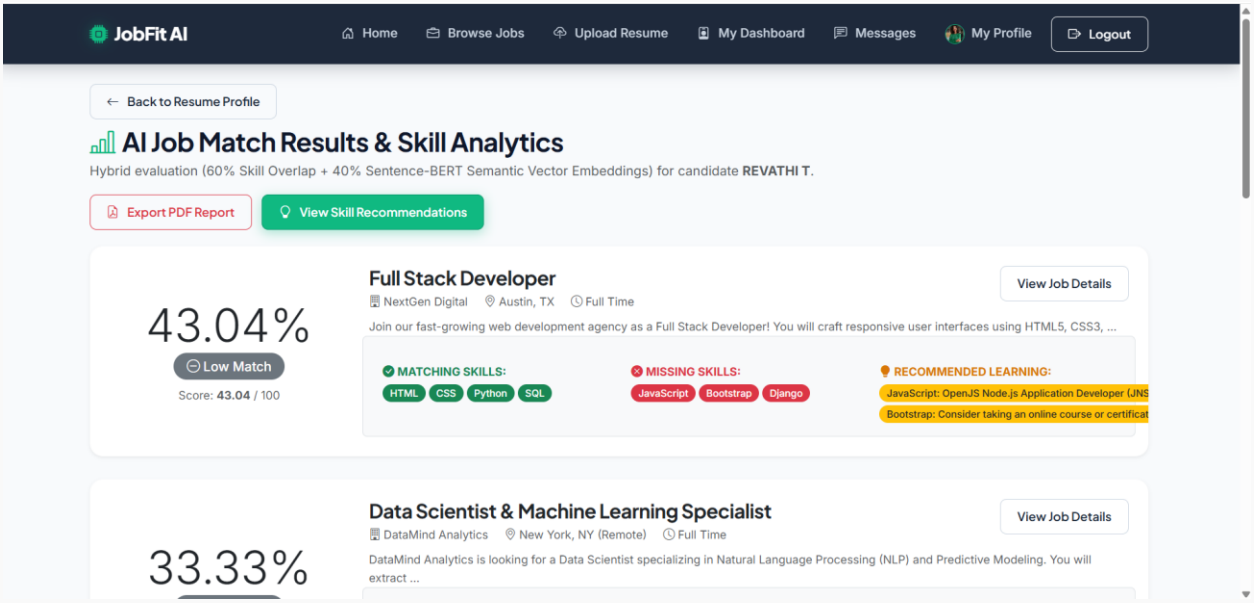


Figure A.3a: AI Matched Jobs

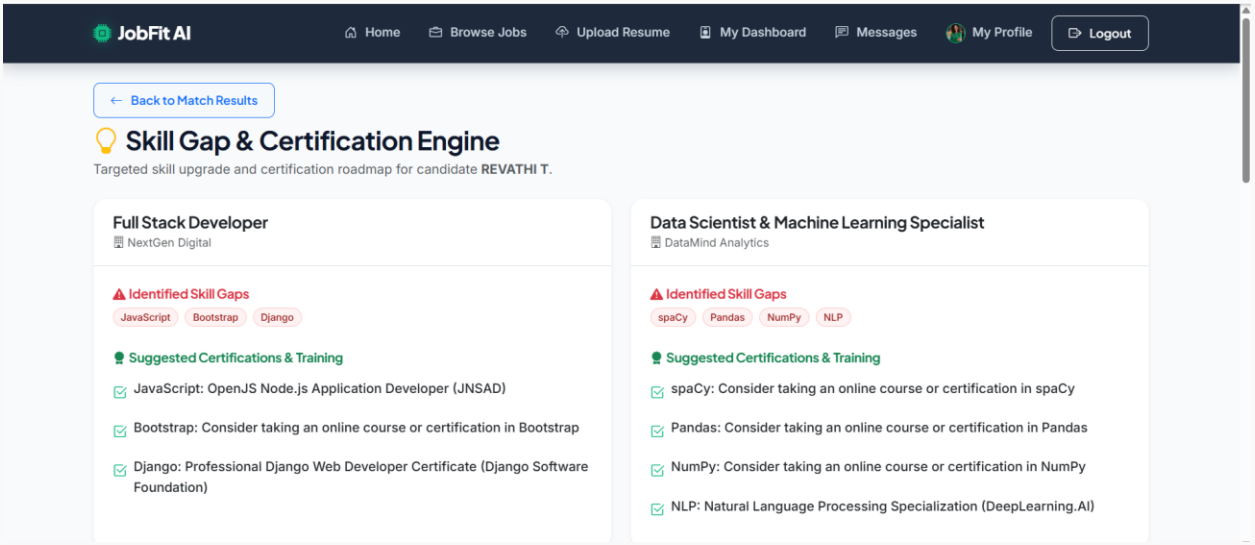


Figure A.3b: Skill Gap Analysis & Course Advisory Page

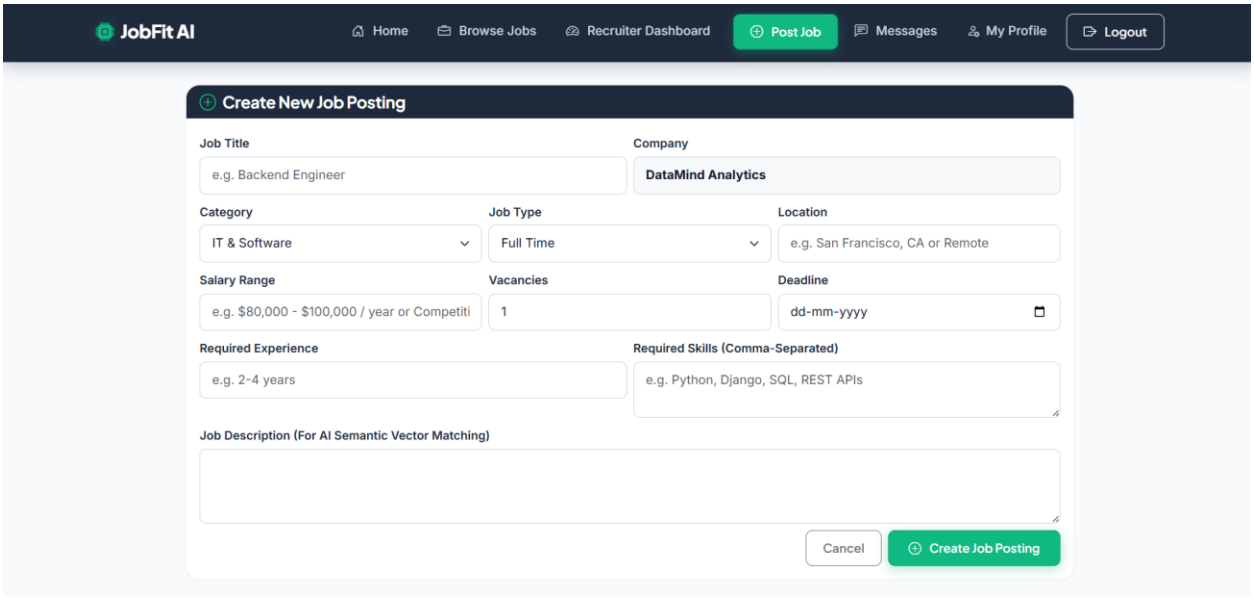


Figure A.4: Recruiter Single-Viewport Job Posting Form

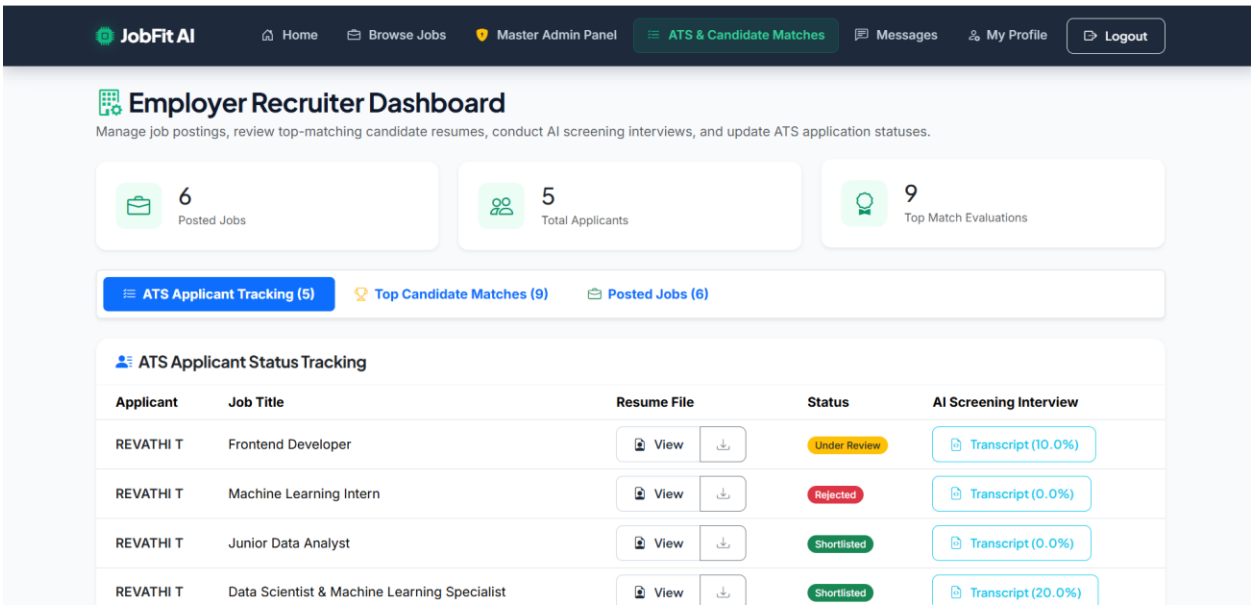


Figure A.5: Recruiter ATS Candidate Leaderboard

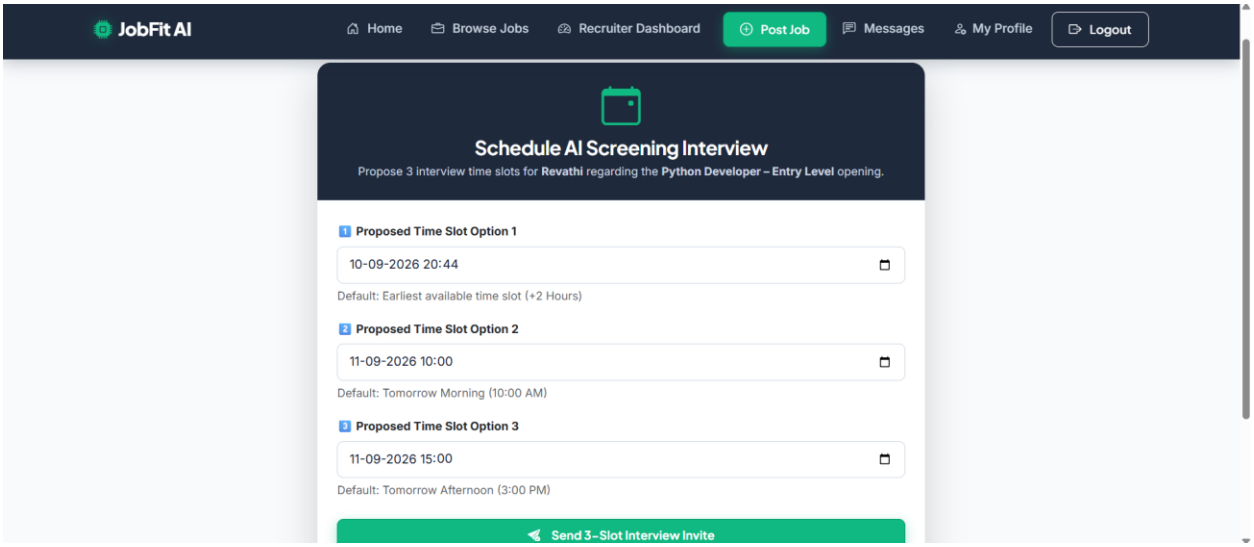


Figure A.6a: Recruiter 3-Slot Interview Proposal Form

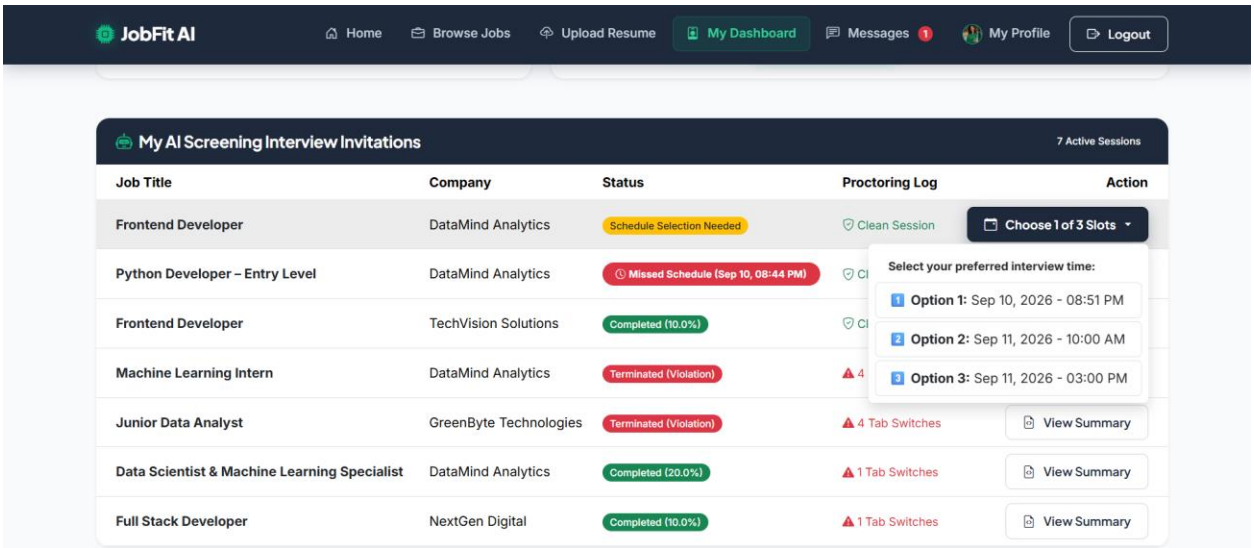


Figure A.6b: Candidate 1-Click Slot Confirmation View

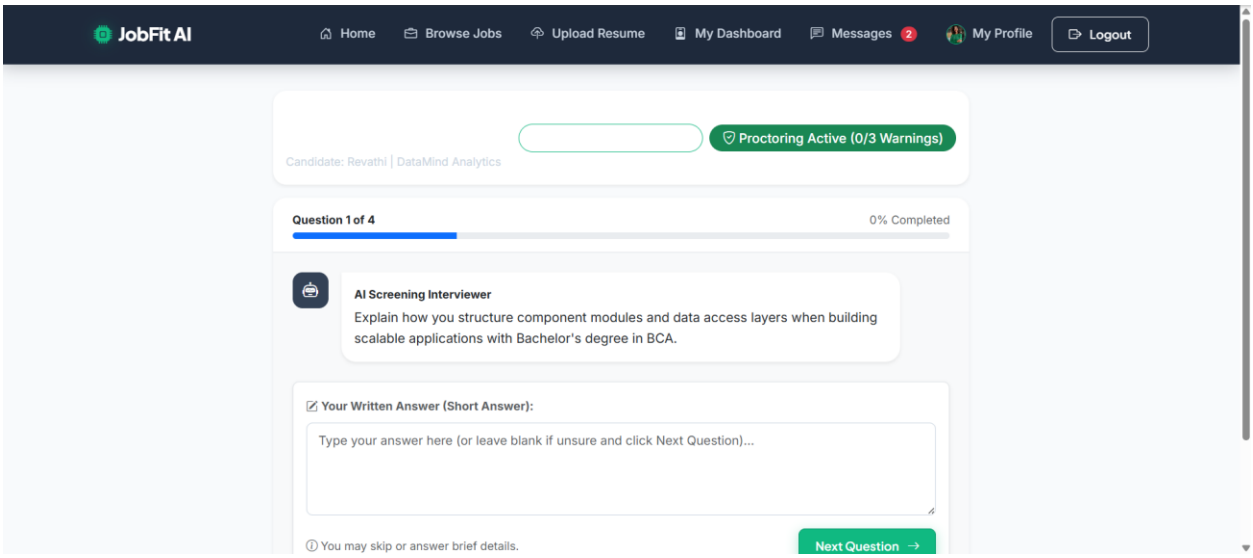


Figure A.7a: Proctored AI Technical Screening Test Room

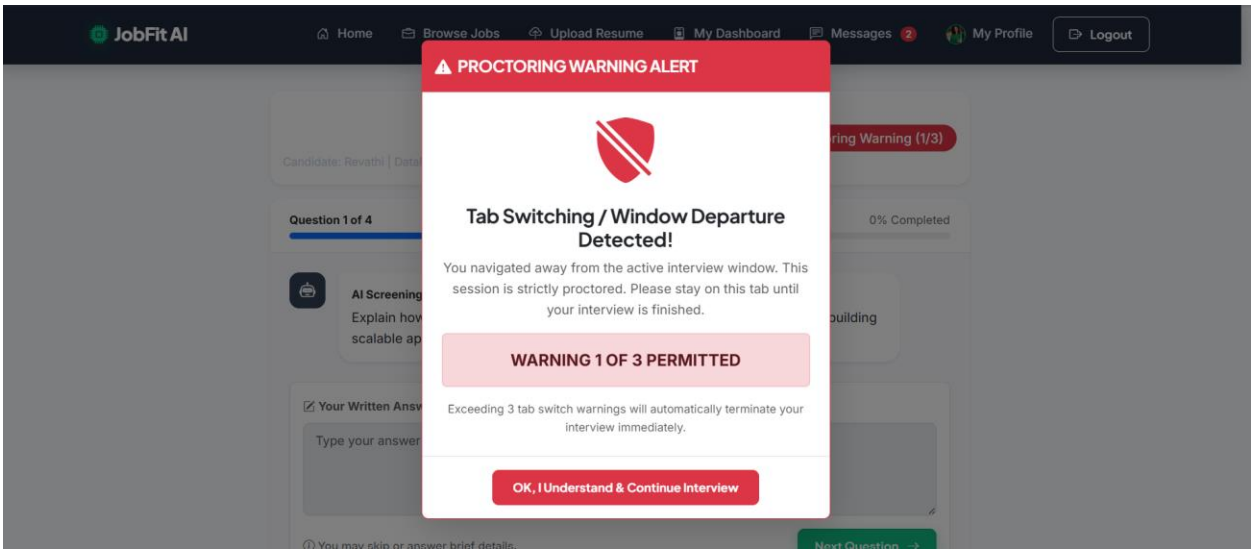


Figure A.7b: Tab-Switch Anti-Cheat Alert Warning Popup

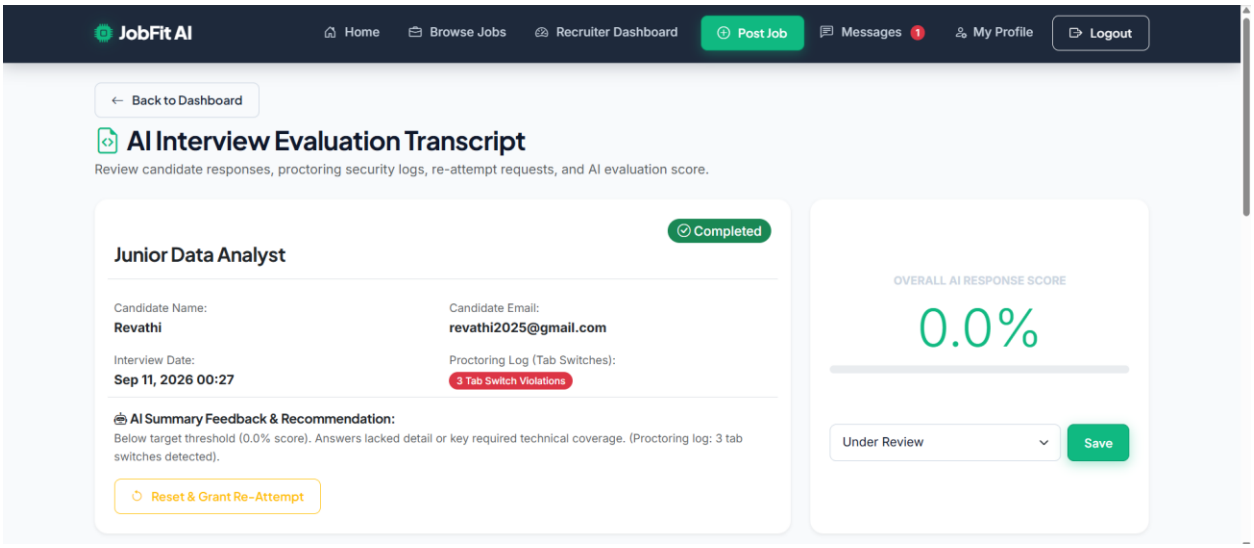


Figure A.8a: Proctored Candidate Evaluation Scorecard

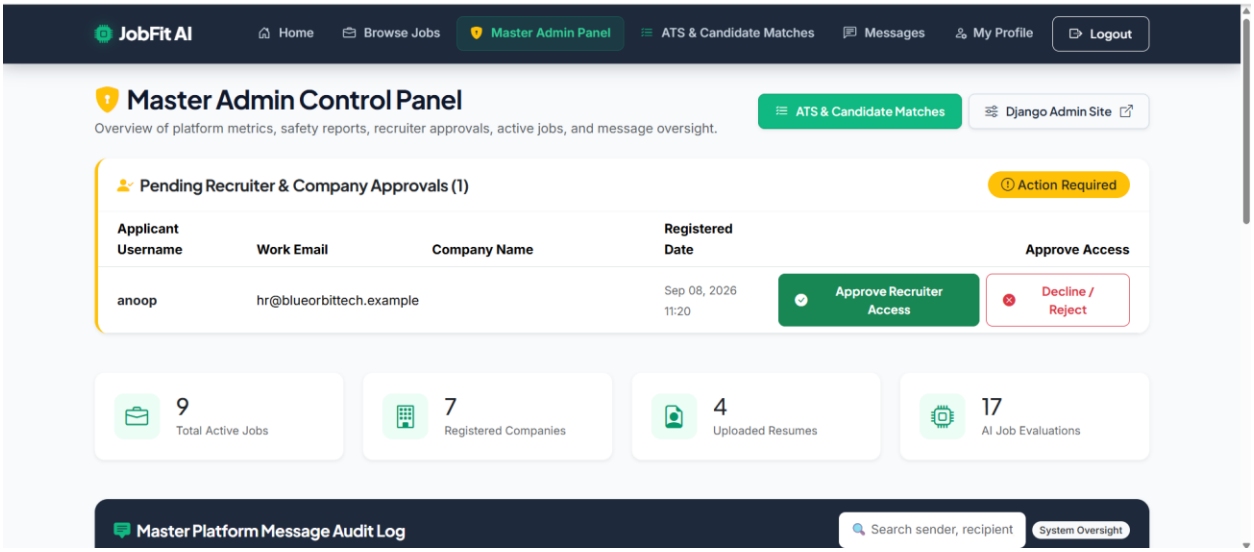


Figure A.8b: Master Admin Recruiter Verification Panel

Appendix B- Sample Code

Models.py

""" JobFit AI - Database Models Definition Defines SQLite3 relational entities for Candidates, Recruiters, Resumes, Jobs, Applications, 3-Slot Interview Scheduling, and Proctored Scorecards. """

from django.db import models from django.contrib.auth.models import User

class RecruiterProfile(models.Model): user = models.OneToOneField(User, on_delete=models.CASCADE, related_name='recruiter_profile')company_name = models.CharField(max_length=255)designation = models.CharField(max_length=150)phone_number = models.CharField(max_length=20)is_verified = models.BooleanField(default=False)created_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return f"{self.user.get_full_name()} ({self.company_name})"

```
class CandidateProfile(models.Model): user = models.OneToOneField(User,
on_delete=models.CASCADE, related_name='candidate_profile')phone_number =
models.CharField(max_length=20, blank=True, null=True)location =
models.CharField(max_length=150, blank=True, null=True)total_experience_years =
models.FloatField(default=0.0)created_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return self.user.get_full_name() or self.user.username

class ResumeDocument(models.Model): candidate = models.ForeignKey(CandidateProfile,
on_delete=models.CASCADE, related_name='resumes')file_path =
models.FileField(upload_to='resumes/')extracted_text = models.TextField(blank=True,
null=True)parsed_skills = models.JSONField(default=list)quality_score =
models.FloatField(default=0.0)  # 0 to 100letter_grade = models.CharField(max_length=5,
default='C')  # A+, A, B, Cuploaded_at = models.DateTimeField(auto_now_add=True)

def __str__(self):
    return f"Resume - {self.candidate.user.username}
({self.uploaded_at.strftime('%Y-%m-%d')})"

class JobPosting(models.Model): recruiter = models.ForeignKey(RecruiterProfile,
on_delete=models.CASCADE, related_name='job_postings')title =
models.CharField(max_length=255)department =
models.CharField(max_length=150)company_name =
models.CharField(max_length=255)description = models.TextField()required_skills =
models.JSONField(default=list)min_experience_years =
models.FloatField(default=0.0)salary_range = models.CharField(max_length=100, blank=True,
null=True)is_active = models.BooleanField(default=True)created_at =
models.DateTimeField(auto_now_add=True)

def __str__(self):
    return f"{self.title} at {self.company_name}"
```

class JobApplication(models.Model): STATUS_CHOICES = (('APPLIED', 'Applied'), ('UNDER_REVIEW', 'Under Review'),   ('SHORTLISTED', 'Shortlisted'),   ('REJECTED', 'Rejected'),   ('HIRED', 'Hired'),)job = models.ForeignKey(JobPosting, on_delete=models.CASCADE, related_name='applications')candidate = models.ForeignKey(CandidateProfile, on_delete=models.CASCADE, related_name='applications')resume = models.ForeignKey(ResumeDocument, on_delete=models.SET_NULL, null=True)match_score = models.FloatField(default=0.0) # Sentence-BERT Cosine Similarity %status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='APPLIED')applied_at = models.DateTimeField(auto_now_add=True)

class Meta:
    unique_together = ('job', 'candidate')

class InterviewSchedule(models.Model): application = models.OneToOneField(JobApplication, on_delete=models.CASCADE, related_name='interview_schedule')slot_1 = models.DateTimeField()slot_2 = models.DateTimeField()slot_3 = models.DateTimeField()confirmed_slot = models.DateTimeField(blank=True, null=True)meeting_link = models.URLField(blank=True, null=True)status = models.CharField(max_length=20, default='PROPOSED') # PROPOSED, CONFIRMED, COMPLETED

def __str__(self):
    return f"Interview for {self.application.candidate.user.username} - {self.status}"

class ProctoredScorecard(models.Model): application = models.OneToOneField(JobApplication, on_delete=models.CASCADE, related_name='scorecard')test_score = models.FloatField(default=0.0) # 0 to 100tab_switch_count = models.IntegerField(default=0)qa_transcript = models.JSONField(default=list)overall_recommendation = models.CharField(max_length=50, default='NEUTRAL')completed_at = models.DateTimeField(auto_now_add=True)

```
def __str__(self):
    return f"Scorecard - {self.application.candidate.user.username} (Score: {self.test_score}%)"
```

Matcher.py

```
""" JobFit AI - Sentence-BERT Vector Matcher Engine Computes semantic Cosine Similarity between Job Descriptions and Candidate Resumes using all-MiniLM-L6-v2 384-dimensional dense vector embeddings. """

import numpy as np from sentence_transformers import SentenceTransformer from sklearn.metrics.pairwise import cosine_similarity

Load lightweight pre-trained Sentence-BERT model once during server startup

model = SentenceTransformer('all-MiniLM-L6-v2')

def calculate_match_score(job_text: str, resume_text: str, job_skills: list, candidate_skills: list) -> float: """Calculates weighted semantic match percentage between candidate resume and job posting.Weights: 70% Semantic Embedding Similarity + 30% Exact Skill Keyword Overlap."""if not job_text or not resume_text:    return 0.0

# 1. Compute 384-dimensional Sentence-BERT embeddings
embeddings = model.encode([job_text, resume_text])
semantic_sim = float(cosine_similarity([embeddings[0]], [embeddings[1]])[0][0])
semantic_score = max(0.0, semantic_sim) * 100.0

# 2. Compute Skill Keyword Overlap Percentage
if job_skills:
    matched_skills = set(job_skills).intersection(set(candidate_skills))
    skill_score = (len(matched_skills) / len(job_skills)) * 100.0
else:
    skill_score = semantic_score
```

3. Hybrid Weighted Score Calculation
final_score = (0.70 * semantic_score) + (0.30 * skill_score)
return round(min(100.0, max(0.0, final_score)), 2)

def identify_skill_gaps(job_skills: list, candidate_skills: list) -> list: """Returns list of missing required skills for candidate learning recommendations."""cand_skills_lower = {s.lower() for s in candidate_skills}missing_skills = [skill for skill in job_skills if skill.lower() not in cand_skills_lower]return missing_skills

resume_parser.py

"""

JobFit AI - Document Parser & Skill Extraction Service

Extracts raw text from PDF/DOCX resumes and identifies technical skills using spaCy PhraseMatcher.

"""

import pdfplumber

import docx

import spacy

from spacy.matcher import PhraseMatcher

Load spaCy English NLP pipeline

nlp = spacy.load("en_core_web_sm")

```
TECHNICAL_SKILL_DATABASE = [
    "Python", "Django", "Flask", "FastAPI", "Java", "Spring Boot", "C++", "C#",
    "JavaScript", "TypeScript", "React", "Node.js", "Vue.js", "Angular", "HTML5", "CSS3",
    "SQL", "PostgreSQL", "MySQL", "SQLite", "MongoDB", "Redis", "Docker", "Kubernetes",
    "AWS", "Azure", "GCP", "Git", "CI/CD", "Machine Learning", "Deep Learning",
    "PyTorch", "TensorFlow", "Scikit-Learn", "NLP", "spaCy", "Pandas", "NumPy"
]


def extract_text_from_file(file_path: str) -> str:
    """Extracts plain text content from PDF or DOCX file."""
    text = ""
    if file_path.endswith('.pdf'):
        with pdfplumber.open(file_path) as pdf:
            for page in pdf.pages:
                page_text = page.extract_text()
                if page_text:
                    text += page_text + "\n"
    elif file_path.endswith('.docx'):
        doc = docx.Document(file_path)
        text = "\n".join([p.text for p in doc.paragraphs if p.text])
    return text.strip()
```

```
def extract_skills_spacy(text: str) -> list:
    """Identifies technical skills in text using spaCy PhraseMatcher."""
    if not text:
        return []

    matcher = PhraseMatcher(nlp.vocab, attr="LOWER")
    patterns = [nlp.make_doc(skill) for skill in TECHNICAL_SKILL_DATABASE]
    matcher.add("SKILL_PATTERNS", patterns)

    doc = nlp(text)
    matches = matcher(doc)

    extracted_skills = set()
    for match_id, start, end in matches:
        extracted_skills.add(doc[start:end].text.title())

    return sorted(list(extracted_skills))
```

```
def calculate_resume_quality_score(extracted_text: str, skills: list) -> tuple:
    """Calculates Quality Score (0-100) and Letter Grade (A+, A, B, C)."""
    score = 0.0
    word_count = len(extracted_text.split())

    # Word count scoring (ideal: 300 - 1000 words)
    if 300 <= word_count <= 1000:
        score += 40.0
    elif word_count > 100:
        score += 25.0

    # Technical skills count scoring
    skill_count = len(skills)
    score += min(40.0, skill_count * 5.0)

    # Structure heuristics
    text_lower = extracted_text.lower()
    if "education" in text_lower:
        score += 10.0
    if "experience" in text_lower or "projects" in text_lower:
        score += 10.0
```

```
    final_score = min(100.0, score)

    if final_score >= 90:
        grade = 'A+'
    elif final_score >= 75:
        grade = 'A'
    elif final_score >= 60:
        grade = 'B'
    else:
        grade = 'C'

    return round(final_score, 1), grade
```

Views.py

```
"""
JobFit AI - Main Django Controllers / Views
Handles Resume Processing, Job Matching Leaderboard, 3-Slot Interview Scheduling,
and Proctored Technical Test Evaluation.
"""

from django.shortcuts import render, redirect, get_object_or_404
```

```
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from .models import JobPosting, JobApplication, ResumeDocument, InterviewSchedule, ProctoredScorecard
from .services.matcher import calculate_match_score, identify_skill_gaps
from .services.resume_parser import extract_text_from_file, extract_skills_spacy, calculate_resume_quality_score


@login_required
def upload_resume_view(request):
    """Candidate resume upload, automated parsing, and scoring controller."""
    if request.method == 'POST' and request.FILES.get('resume_file'):
        uploaded_file = request.FILES['resume_file']
        candidate_profile = request.user.candidate_profile

        # Save resume document
        resume_doc = ResumeDocument.objects.create(
            candidate=candidate_profile,
            file_path=uploaded_file
        )

        # Parse text & skills
```

```
            extracted_text = extract_text_from_file(resume_doc.file_path.path)
            skills = extract_skills_spacy(extracted_text)
            quality_score, grade = calculate_resume_quality_score(extracted_text, skills)

            resume_doc.extracted_text = extracted_text
            resume_doc.parsed_skills = skills
            resume_doc.quality_score = quality_score
            resume_doc.letter_grade = grade
            resume_doc.save()

            return redirect('candidate_dashboard')

    return render(request, 'candidate/upload_resume.html')


@login_required
def recruiter_ats_leaderboard_view(request, job_id):
    """Recruiter ATS leaderboard listing candidates sorted by AI Match Percentage."""
    job = get_object_or_404(JobPosting, id=job_id, recruiter=request.user.recruiter_profile)
    applications = JobApplication.objects.filter(job=job).order_by('-match_score')
```

```
    return render(request, 'recruiter/ats_leaderboard.html', {
        'job': job,
        'applications': applications
    })


@login_required
def confirm_interview_slot_view(request, schedule_id):
    """Candidate 1-click 3-slot interview timing selection controller."""
    schedule = get_object_or_404(InterviewSchedule, id=schedule_id,
application__candidate=request.user.candidate_profile)

    if request.method == 'POST':
        selected_slot = request.POST.get('selected_slot')
        if selected_slot in ['slot_1', 'slot_2', 'slot_3']:
            schedule.confirmed_slot = getattr(schedule, selected_slot)
            schedule.status = 'CONFIRMED'
            schedule.save()
            return JsonResponse({'status': 'success', 'confirmed_slot':
schedule.confirmed_slot.strftime('%Y-%m-%d %H:%M')})

    return render(request, 'candidate/confirm_interview.html', {'schedule': schedule})
```

```
@login_required
def submit_proctored_test_view(request, application_id):
    """Processes proctored test answers and tab-switch anti-cheat logs."""
    application = get_object_or_404(JobApplication, id=application_id,
candidate=request.user.candidate_profile)

    if request.method == 'POST':
        tab_switches = int(request.POST.get('tab_switch_count', 0))
        answers = request.POST.getlist('answers[]')

        # Calculate screening score (e.g. static key evaluation heuristic)
        score = max(0.0, 85.0 - (tab_switches * 10.0))

        scorecard = ProctoredScorecard.objects.create(
            application=application,
            test_score=score,
            tab_switch_count=tab_switches,
            qa_transcript=answers,
            overall_recommendation='HIGHLY_RECOMMENDED' if score >= 75 else 'REJECT'
```

```
)


return JsonResponse({'status': 'completed', 'score': score, 'scorecard_id': scorecard.id})
```